

C++ Standard Library Issues to be moved in Prague

Doc. no. P2051R0

Date: Revised 2020-01-13 at 07:03:57 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[3194](#)⁽ⁱ⁾. `ConvertibleTo` prose does not match code

Section: 18.4.4 [\[concept.convertible\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hubert Tong **Opened:** 2019-03-05 **Last modified:** 2020-01-12

Priority: 1

View other [active issues](#) in `[concept.convertible]`.

View all other [issues](#) in `[concept.convertible]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

The prose in [N4800](#) subclause `[concept.convertibleto]` indicates that the requirement is for an expression of a particular type and value category to be both implicitly and explicitly convertible to some other type. However, for a type

```
struct A { A(const A&) = delete; };
```

`ConvertibleTo<const A, A>` would be false despite the following being okay:

```
const A f();

A test() {
    static_cast<A>(f());
    return f();
}
```

[2019-03-15 Priority set to 1 after reflector discussion]

[2019-07-14 Tim adds PR based on discussion in 2019-07-09 LWG telecon]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4820](#), and also resolves LWG [3151](#).

1. Modify `[concept.convertibleto]` as indicated:

-1- The `ConvertibleTo` concept requires **an rvalue** expression of a particular type and value category to be both implicitly and explicitly convertible to some other type. The implicit and explicit conversions are required to produce equal results.

```
template<class From, class To>
concept ConvertibleTo =
    is_convertible_v<From, To> &&
    requires(add_rvalue_reference_t<From> (&f)()) {
        static_cast<To>(f());
    };
```

-2- Let test be the invented function:

```
To test(add_rvalue_reference_t<From> (&f)()) {
    return f();
}
```

for some types From and To, and let `f` be a function with no arguments and return type `add_rvalue_reference_t<From>` such that `f()` is equality-preserving. From and To model `ConvertibleTo<From, To>` only if:

(2.1) — To is not an object or reference-to-object type, or `static_cast<To>(f())` is equal to `test(f)`.

(2.2) — `add_rvalue_reference_t<From>` is not a reference-to-object type, or

(2.2.1) — If `add_rvalue_reference_t<From>` is an rvalue reference to a non const-qualified type, the resulting state of the object referenced by `f()` after either above expression is valid but unspecified (16.5.5.16 [[lib.types.movedfrom](#)]).

(2.2.2) — Otherwise, the object referred to by `f()` is not modified by either above expression.

[2019-09-23; Daniel adjusts wording to working draft changes]

Due to the concept renaming caused by [P1754R1](#) the proposed wording is outdated and needs adjustments.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#), and also resolves LWG [3151](#).

1. Modify 18.4.4 [[concept.convertible](#)] as indicated:

-1- The `convertible_to` concept requires ~~an~~ `glvalue` expression of a particular type and value category to be both implicitly and explicitly convertible to some other type. The implicit and explicit conversions are required to produce equal results.

```
template<class From, class To>
concept convertible_to =
    is_convertible_v<From, To> &&
    requires(add_rvalue_reference_t<From> (&f)()) {
        static_cast<To>(f());
    };

```

-2- Let `test` be the invented function:

```
To test(add_rvalue_reference_t<From> (&f)()) {
    return f();
}

```

for some types From and To, and let `f` be a function with no arguments and return type `add_rvalue_reference_t<From>` such that `f()` is equality-preserving. From and To model `convertible_to<From, To>` only if:

(2.1) — To is not an object or reference-to-object type, or `static_cast<To>(f())` is equal to `test(f)`.

(2.2) — `add_rvalue_reference_t<From>` is not a reference-to-object type, or

(2.2.1) — If `add_rvalue_reference_t<From>` is an rvalue reference to a non const-qualified type, the resulting state of the object referenced by `f()` after either above expression is valid but unspecified (16.5.5.16 [[lib.types.movedfrom](#)]).

(2.2.2) — Otherwise, the object referred to by `f()` is not modified by either above expression.

[2019-11-06 Tim updates PR based on discussion in Belfast LWG evening session]

"glvalue" is incorrect because we want to allow testing `convertible_to<void, void>`. It's also less than clear how the "expression" and "a particular type" in the first sentence correspond to the parameters of the concept.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4835](#), and also resolves LWG [3151](#).

1. Modify 18.4.4 [[concept.convertible](#)] as indicated:

-1- The convertible_to concept ~~for types From and To~~ requires an expression ~~E such that decltype((E)) is add rvalue reference t<From> of a particular type and value category~~ to be both implicitly and explicitly convertible to ~~some other type To~~. The implicit and explicit conversions are required to produce equal results.

```
template<class From, class To>
concept convertible_to =
    is_convertible_v<From, To> &&
    requires(add_rvalue_reference_t<From> (&f)()) {
        static_cast<To>(f());
    };
```

-2- Let ~~FromR be add rvalue reference t<From> and~~ test be the invented function:

```
To test(FromR (&f)()) {
    return f();
}
```

for some types From and To, and let f be a function with no arguments and return type FromR such that f() is equality-preserving. From and To model convertible_to<From, To> only if:

(2.1) — To is not an object or reference-to-object type, or static_cast<To>(f()) is equal to test(f).

(2.2) — FromR is not a reference-to-object type, or

(2.2.1) — If FromR is an rvalue reference to a non const-qualified type, the resulting state of the object referenced by f() after either above expression is valid but unspecified (16.5.5.16 [\[lib.types.movedfrom\]](#)).

(2.2.2) — Otherwise, the object referred to by f() is not modified by either above expression.

[2019-11-09 Tim rephrased first sentence based on discussion in Belfast LWG Saturday session]

[Status to Tentatively ready after Belfast LWG Saturday session]

Proposed resolution:

This wording is relative to [N4835](#), and also resolves LWG [3151](#).

1. Modify 18.4.4 [\[concept.convertible\]](#) as indicated:

-1- ~~Given types From and To and an expression E such that decltype((E)) is add rvalue reference t<From>, convertible to<From, To> The convertible_to concept~~ requires ~~E an expression of a particular type and value category~~ to be both implicitly and explicitly convertible to ~~some other type To~~. The implicit and explicit conversions are required to produce equal results.

```
template<class From, class To>
concept convertible_to =
    is_convertible_v<From, To> &&
    requires(add_rvalue_reference_t<From> (&f)()) {
        static_cast<To>(f());
    };
```

-2- Let ~~FromR be add rvalue reference t<From> and~~ test be the invented function:

```
To test(FromR (&f)()) {
    return f();
}
```

for some types From and To, and let f be a function with no arguments and return type FromR such that f() is equality-preserving. From and To model convertible_to<From, To> only if:

(2.1) — To is not an object or reference-to-object type, or static_cast<To>(f()) is equal to test(f).

(2.2) — FromR is not a reference-to-object type, or

(2.2.1) — If FromR is an rvalue reference to a non const-qualified type, the resulting state of the object referenced by f() after either above expression is valid but unspecified (16.5.5.16 [\[lib.types.movedfrom\]](#)).

(2.2.2) — Otherwise, the object referred to by f() is not modified by either above expression.

3233⁽ⁱ⁾. Broken requirements for shared_ptr converting constructors

Section: 20.11.3.1 [[util.smartptr.shared.const](#)] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-07-10 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [[util.smartptr.shared.const](#)].

View all other [issues](#) in [[util.smartptr.shared.const](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

Issue [2875](#) added 20.11.3.1 [[util.smartptr.shared.const](#)] paragraph 13:

Remarks: When T is an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and either T is $U[N]$ and $Y(*)[N]$ is convertible to T^* , or T is $U[]$ and $Y(*)[]$ is convertible to T^* . When T is not an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and Y^* is convertible to T^* .

which pertains to the four constructor overloads:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template<class D> shared_ptr(nullptr_t p, D d);
template<class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

which is fine (ignoring for now that two occurrences of "this constructor" should read "these constructors") for the two overloads with a template parameter Y , but not so much for the two with no such template parameter.

MSVC ignores the constraints on Y for the overloads with no such template parameter, whereas libstdc++ and libc++ seem to ignore *all* of the constraints for those overloads (See [Compiler Explorer](#)). We should fix the broken wording, ideally by requiring the MSVC interpretation - the `nullptr_t` constructors participate in overload resolution only when `is_movable_v<D>` is true and `d(p)` is well-formed - so concepts and traits that check constructibility are correct.

[2019-11-17 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 20.11.3.1 [[util.smartptr.shared.const](#)] as indicated:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template<class D> shared_ptr(nullptr_t p, D d);
template<class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-?- Constraints: `is_move_constructible_v<D>` is true, and `d(p)` is a well-formed expression. For the first two overloads:

- If T is an array type, then either T is $U[N]$ and $Y(*)[N]$ is convertible to T^* , or T is $U[]$ and $Y(*)[]$ is convertible to T^* .
- If T is not an array type, then Y^* is convertible to T^* .

-9- Requires/Expects: Construction of `d` and a deleter of type [...]

[...]

~~**-13- Remarks:** When T is an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and either T is $U[N]$ and $Y(*)[N]$ is convertible to T^* , or T is $U[]$ and $Y(*)[]$ is convertible to T^* . When T is not an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and Y^* is convertible to T^* .~~

3254⁽ⁱ⁾. Strike stop_token's operator !=

Section: 32.3.3 [\[stoptoken\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2019-08-06 Last modified: 2020-01-12

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

In the shiny new world of the future, we need not declare overloads of `operator!=` that return `!(x == y)`; the rewrite rule in 12.4.1.2 [\[over.match.oper\]](#) para 3.4.3 achieves the same effect. Consequently, we should not declare such unnecessary `operator!=` overloads. The synopsis of class `stop_token` in 32.3.3 [\[stoptoken\]](#) declares exactly such an `operator!=` friend which should be struck.

[01-2020 Moved to Tentatively Ready after 5 positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 32.3.3 [\[stoptoken\]](#), class `stop_token` synopsis, as indicated:

```
namespace std {
    class stop_token {
    public:
        [...]
        [[nodiscard]] friend bool operator==(const stop_token& lhs, const stop_token& rhs) noexcept;
        [[nodiscard]] friend bool operator!=(const stop_token& lhs, const stop_token& rhs) noexcept;
        friend void swap(stop_token& lhs, stop_token& rhs) noexcept;
    };
}
```

2. Modify 32.3.3.3 [\[stoptoken.cmp\]](#) and 32.3.3.4 [\[stoptoken.special\]](#) as indicated:

32.3.3.3 [Non-member functions](#) [Comparisons](#) [\[stoptoken.nonmemberscmp\]](#)

```
[[nodiscard]] bool operator==(const stop_token& lhs, const stop_token& rhs) noexcept;
```

-1- *Returns:* true if lhs and rhs have ownership of the same stop state or if both lhs and rhs do not have ownership of a stop state; otherwise false.

```
[[nodiscard]] bool operator!=(const stop_token& lhs, const stop_token& rhs) noexcept;
```

~~-2- *Returns:* !(lhs==rhs);~~

32.3.3.4 [Specialized algorithms](#) [\[stoptoken.special\]](#)

```
friend void swap(stop_token& x, stop_token& y) noexcept;
```

-1- *Effects:* Equivalent to: `x.swap(y)`.

[3264^{\(i\)}](#). `disable_sized_range` and `ranges::size` redundantly use `disable_sized_range`

Section: 24.4.3 [\[range.sized\]](#) Status: [Ready](#) Submitter: Casey Carter Opened: 2019-08-26 Last modified: 2019-12-02

Priority: 1

View all issues with [Ready](#) status.

Discussion:

`disable_sized_range` (24.4.3 [\[range.sized\]](#)) is an opt-out trait that users may specialize when their range type conforms to the syntax of `sized_range` but not its semantics, or when the type is so poorly suited to the Standard Library that even testing the validity of the expressions `r.size()` or `size(r)` for a range `r` is impossible. The library inspects `disable_sized_range` in two places. (1) In the definition of the `sized_range` concept:

```
template<class T>
concept sized_range =
    range<T> &&
    !disable_sized_range<remove_cvref_t<T>> &&
    requires(T& t) { ranges::size(t); };
```

If the pertinent specialization of `disable_sized_range` is true, we avoid checking the validity of the expression `ranges::size(t)` in the *requires-expression*. (2) In the definition of the `ranges::size` CPO itself (24.3.9 [\[range.prim.size\]](#)), the validity of the expressions *decay-*

`copy(E.size())` and `decay-copy(size(E))` is not checked if the pertinent specialization of `disable_sized_range` is true.

`disable_sized_range` is effectively checked *twice* when evaluating `sized_range`. This redundancy could be forgiven, if it did not permit the existence of non-`sized_range`s for which `ranges::size` returns a valid size:

```
struct mytype {};  
using A = mytype[42];  
  
template <>  
constexpr bool std::ranges::disable_sized_range<A> = true;  
  
static_assert(std::ranges::range<A>);  
static_assert(!std::ranges::sized_range<A>);  
static_assert(std::ranges::size(A{}) == 42);  
  
struct myrange {  
    constexpr int* begin() const { return nullptr; }  
    constexpr int* end() const { return nullptr; }  
};  
  
template <>  
constexpr bool std::ranges::disable_sized_range<myrange> = true;  
  
static_assert(std::ranges::range<myrange>);  
static_assert(!std::ranges::sized_range<myrange>);  
static_assert(std::ranges::size(myrange{}) == 0);
```

We should remove this gap between `ranges::size` and `sized_range` by checking `disable_sized_range` only in the definition of `ranges::size`, and continuing to rely on the validity of `ranges::size` in the `sized_range` concept.

[2019-09-14 Priority set to 1 based on reflector discussion]

[2019-11 Wednesday night issue processing in Belfast.]

Status to Ready

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.4.3 [\[range.sized\]](#) as follows:

```
template<class T>  
concept sized_range =  
    range<T> &&  
    !disable_sized_range<remove_cvref_t<T>> &&  
    requires(T& t) { ranges::size(t); };
```

[3280^{\(i\)}](#). View converting constructors can cause constraint recursion and are unneeded

Section: 24.7.4.2 [\[range.filter.view\]](#), 24.7.5.2 [\[range.transform.view\]](#), 24.7.6.2 [\[range.take.view\]](#), 24.7.10.2 [\[range.join.view\]](#), 24.7.11.2 [\[range.split.view\]](#), 24.7.14.2 [\[range.reverse.view\]](#) **Status:** [Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-09 **Last modified:** 2020-01-12

Priority: 1

View all issues with [Ready](#) status.

Discussion:

The following program fails to compile:

```
#include <ranges>  
  
int main() {  
    namespace ranges = std::ranges;  
    int a[] = {1, 7, 3, 6, 5, 2, 4, 8};  
    auto r0 = ranges::view::reverse(a);  
    auto is_even = [](int i) { return i % 2 == 0; };  
    auto r1 = ranges::view::filter(r0, is_even);  
    int sum = 0;  
    for (auto i : r1) {  
        sum += i;  
    }  
    return sum - 20;  
}
```

The problem comes from constraint recursion, caused by the following constructor:

```
template<viewable_range R>
requires bidirectional_range<R> && constructible_from<V, all_view<R>>
constexpr explicit reverse_view(R&& r);
```

This constructor owes its existence to class template argument deduction; it is the constructor we intend to use to resolve `reverse_view{r}`, which (in accordance to the deduction guide) will construct an object of type `reverse_view<all_view<decltype(r)>>`.

However, we note that `all_view<R>` is always one of:

- `decay_t<R>`
- `ref_view<remove_reference_t<R>>`
- `subrange<iterator_t<R>, sentinel_t<R>, [sized?]>`

In all cases, there is a conversion from `r` to the destination type. As a result, the following non-template `reverse_view` constructor can fulfill the duty that the above constructor was meant to fulfill, and does *not* cause constraint recursion:

```
constexpr explicit reverse_view(V r);
```

In short, the problematic constructor can simply be removed with no negative impact on the design. And the similar constructors from the other range adaptors should similarly be stricken.

Suggested priority P1. The view types are unusable without this change.

This proposed resolution has been implemented in range-v3 and has been shipping for some time.

[2019-10 Priority set to 1 after reflector discussion]

[2019-10 Status set to ready Wednesday night discussion in Belfast.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.7.4.2 [\[range.filter.view\]](#) as indicated:

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
  requires view<V> && is_object_v<Pred>
  class filter_view : public view_interface<filter_view<V, Pred>> {
  [...]
  public:
    filter_view() = default;
    constexpr filter_view(V base, Pred pred);
template<input_range R>
requires viewable_range<R> && constructible_from<V, all_view<R>>
constexpr filter_view(R&& r, Pred pred);
  [...]
  };
  [...]
}

[...]
```

```
template<input_range R>
requires viewable_range<R> && constructible_from<V, all_view<R>>
constexpr filter_view(R&& r, Pred pred);
```

~~-2- Effects: Initializes base_ with views::all(std::forward<R>(r)) and initializes pred_ with std::move(pred);~~

2. Modify 24.7.5.2 [\[range.transform.view\]](#) as indicated:

```
namespace std::ranges {
  template<input_range V, copy_constructible F>
  requires view<V> && is_object_v<F> &&
  regular_invocable<F&, range_reference_t<V>>
  class transform_view : public view_interface<transform_view<V, F>> {
  private:
  [...]
  public:
    transform_view() = default;
    constexpr transform_view(V base, F fun);
template<input_range R>
```

```

requires viewable_range<R> && constructible_from<V, all_view<R>>
constexpr transform_view(R&& r, F fun);
    [...]
};
    [...]
}

[...]
```

```

template<input_range R>
requires viewable_range<R> && constructible_from<V, all_view<R>>
constexpr transform_view(R&& r, F fun);
```

~~-2- Effects: Initializes base_ with views::all(std::forward<R>(r)) and fun_ with std::move(fun).~~

3. Modify 24.7.6.2 [\[range.take.view\]](#) as indicated:

```

namespace std::ranges {
    template<view V>
    class take_view : public view_interface<take_view<V>> {
    private:
        [...]
    public:
        take_view() = default;
        constexpr take_view(V base, range_difference_t<V> count);
template<viewable_range R>
requires constructible_from<V, all_view<R>>
constexpr take_view(R&& r, range_difference_t<V> count);
        [...]
    };
    [...]
}

[...]
```

```

template<viewable_range R>
requires constructible_from<V, all_view<R>>
constexpr take_view(R&& r, range_difference_t<V> count);
```

~~-2- Effects: Initializes base_ with views::all(std::forward<R>(r)) and count_ with count.~~

4. Modify 24.7.10.2 [\[range.join.view\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
        (is_reference_v<range_reference_t<V>> ||
         view_range_value_t<V>>)
    class join_view : public view_interface<join_view<V>> {
    private:
        [...]
    public:
        join_view() = default;
        constexpr explicit join_view(V base);
template<input_range R>
requires viewable_range<R> && constructible_from<V, all_view<R>>
constexpr explicit join_view(R&& r);
        [...]
    };
    [...]
}

[...]
```

```

template<input_range R>
requires viewable_range<R> && constructible_from<V, all_view<R>>
constexpr explicit join_view(R&& r);
```

~~-2- Effects: Initializes base_ with views::all(std::forward<R>(r)).~~

5. Modify 24.7.11.2 [\[range.split.view\]](#) as indicated:

```

namespace std::ranges {
    [...]
    template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
        indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
        (forward_range<V> || tiny_range<Pattern>)
    class split_view : public view_interface<split_view<V, Pattern>> {
    private:
        [...]
    }
```



```

public:
    split_view() = default;
    constexpr split_view(V base, Pattern pattern);
template<input_range R, forward_range P>
requires constructible_from<V, all_view<R>> &&
constructible_from<Pattern, all_view<P>>
    constexpr split_view(R&& r, P&& p);
    [...]
};
[...]
```

```

[...]
```

```

template<input_range R, forward_range P>
requires constructible_from<V, all_view<R>> &&
constructible_from<Pattern, all_view<P>>
    constexpr split_view(R&& r, P&& p);
```

~~-2- Effects: Initializes base_ with views::all(std::forward<R>(r)), and pattern_ with views::all(std::forward<P>(p)).~~

6. Modify 24.7.14.2 [\[range.reverse.view\]](#) as indicated:

```

namespace std::ranges {
    template<view V>
        requires bidirectional_range<V>
        class reverse_view : public view_interface<reverse_view<V>> {
        private:
            [...]
        public:
            reverse_view() = default;
            constexpr explicit reverse_view(V r);
template<viewable_range R>
requires bidirectional_range<R> && constructible_from<V, all_view<R>>
constexpr explicit reverse_view(R&& r);
            [...]
        };
    [...]
}
```

```

[...]
```

```

template<viewable_range R>
requires bidirectional_range<R> && constructible_from<V, all_view<R>>
constexpr explicit reverse_view(R&& r);
```

~~-2- Effects: Initializes base_ with views::all(std::forward<R>(r)).~~

3281⁽ⁱ⁾. Conversion from *pair-like* types to subrange is a silent semantic promotion

Section: 24.5.3 [\[range.subrange\]](#) Status: [Ready](#) Submitter: Eric Niebler Opened: 2019-09-10 Last modified: 2019-12-02

Priority: 1

View other [active issues](#) in [\[range.subrange\]](#).

View all other [issues](#) in [\[range.subrange\]](#).

View all issues with [Ready](#) status.

Discussion:

Just because a pair is holding two iterators, it doesn't mean those two iterators denote a valid range. Implicitly converting such *pair-like* types to a subrange is dangerous and should be disallowed.

Suggested priority P1.

Implementation experience: range-v3's subrange lacks these constructors.

[2019-10 Priority set to 1 and status to LEWG after reflector discussion]

[2019-11 Status to Ready after LWG discussion Friday in Belfast.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.5.3 [\[range.subrange\]](#) as indicated:

```
namespace std::ranges {
[...]  
    template<class T, class U, class V>  
    concept pair_like_convertible_to = // exposition only  
    !range<T> && pair_like_remove_reference_t<T> &&  
    requires(T&& t) {  
        { get<0>(std::forward<T>(t)) } -> convertible_to<U>;  
        { get<1>(std::forward<T>(t)) } -> convertible_to<V>;  
    };  
[...]  
    template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K =  
        sized_sentinel_for<S, I> ? subrange_kind::sized : subrange_kind::unsized>  
    requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)  
    class subrange : public view_interface<subrange<I, S, K>> {  
    private:  
    [...]  
    public:  
    [...]  
    template<not same as<subrange> PairLike>  
    requires pair_like_convertible_to<PairLike, I, S>  
    constexpr subrange(PairLike&& r) requires (!StoreSize)  
        : subrange{std::get<0>(std::forward<PairLike>(r)),  
                   std::get<1>(std::forward<PairLike>(r))}  
    {}  
    template<pair_like_convertible_to<I, S> PairLike>  
    constexpr subrange(PairLike&& r, make_unsigned_like_t(iter_difference_t<I>) n)  
        requires (K == subrange_kind::sized)  
        : subrange{std::get<0>(std::forward<PairLike>(r)),  
                   std::get<1>(std::forward<PairLike>(r)), n}  
    {}  
    [...]  
};  
[...]  
}
```

[3284^{\(i\)}](#). [random_access_iterator](#) semantic constraints accidentally promote difference type using unary negate

Section: 23.3.4.13 [\[iterator.concept.random.access\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-10 **Last modified:** 2019-12-02

Priority: 0

View all other [issues](#) in [\[iterator.concept.random.access\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

23.3.4.13 [\[iterator.concept.random.access\]](#)/p2.7 says:

(b += -n) is equal to a

Unary minus can do integer promotion. That is not the intent here.

[2019-10-12 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 23.3.4.13 [\[iterator.concept.random.access\]](#) as indicated:

-2- Let a and b be valid iterators of type I such that b is reachable from a after n applications of ++a, let D be iter_difference_t<I>, and let n denote a value of type D. I models random_access_iterator only if:

(2.1) — (a += n) is equal to b.

[...]

(2.7) — (b += `D(-n)`) is equal to a.

[...]

[3285^{\(i\)}](#). The type of a customization point object shall satisfy semiregular

Section: 16.4.2.2.6 [\[customization.point.object\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-10 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

We should be testing the un-cv-qualified type of a customization point object.

[2019-10-12 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 16.4.2.2.6 [\[customization.point.object\]](#) as indicated:

-2- The type of a customization point object `ignoring cv-qualifiers` shall model semiregular (18.6 [\[concepts.object\]](#)).

[3286^{\(i\)}](#). `ranges::size` is not required to be valid after a call to `ranges::begin` on an input range

Section: 24.7.6.2 [\[range.take.view\]](#), 24.5.3.1 [\[range.subrange.ctor\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-10 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

On an input (but not forward) range, `begin(rng)` is not required to be an equality-preserving expression (24.4.2 [\[range.range\]](#)/3.3). If the range is *also* sized, then it is not valid to call `size(rng)` after `begin(rng)` (24.4.3 [\[range.sized\]](#)/2.2). In several places in the ranges clause, this precondition is violated. A trivial re-expression of the effects clause fixes the problem.

[2019-10-12 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.7.6.2 [\[range.take.view\]](#), class template `take_view` synopsis, as indicated:

```
namespace std::ranges {
    template<view V>
    class take_view : public view_interface<take_view<V>> {
    private:
        [...]
    public:
        [...]
        constexpr auto begin() requires (!simple-view<V>) {
            if constexpr (sized_range<V>) {
                if constexpr (random_access_range<V>)
                    return ranges::begin(base_);
                else {
                    auto sz = size();
                    return counted_iterator{ranges::begin(base_), size(), sz};
                }
            } else
                return counted_iterator{ranges::begin(base_), count_};
        }
    };
}
```

```

    }

    constexpr auto begin() const requires range<const V> {
        if constexpr (sized_range<const V>) {
            if constexpr (random_access_range<const V>)
                return ranges::begin(base_);
            else {
                auto sz = size();
                return counted_iterator{ranges::begin(base_), size()sz};
            }
        } else
            return counted_iterator{ranges::begin(base_), count_};
    }

    [...]
};
[...]
```

2. Modify 24.5.3.1 [\[range.subrange.ctor\]](#) as indicated:

```

template<not-same-as<subrange> R>
requires forwarding-range<R> &&
    convertible_to<iterator_t<R>, I> && convertible_to<sentinel_t<R>, S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);
```

-6- *Effects*: Equivalent to:

(6.1) — If StoreSize is true, subrange{~~ranges::begin(r), ranges::end(r)~~r, ranges::size(r)}.

(6.2) — Otherwise, subrange{ranges::begin(r), ranges::end(r)}.

3291⁽ⁱ⁾. `iota_view::iterator` has the wrong `iterator_category`

Section: 24.6.3.3 [\[range.iota.iterator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Eric Niebler **Opened:** 2019-09-13 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

In the old way of looking at the world, forward iterators need to return real references. Since dereferencing `iota_view`'s iterators returns by value, it cannot be a C++17 forward iterator. (It can, however, be a C++20 `forward_iterator`.) However, `iota_view`'s iterator has an `iterator_category` that (sometimes) falsely claims that it is forward or better (depending on the properties of the `weakly_incrementable` type it wraps).

[2019-10-19 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.6.3.3 [\[range.iota.iterator\]](#) as indicated:

```

namespace std::ranges {
    template<class W, class Bound>
    struct iota_view<W, Bound>::iterator {
    private:
        [...]
    public:
        using iterator_conceptcategory = see below;
        using iterator_category = input_iterator_tag;
        using value_type = W;
        using difference_type = IOTA-DIFF-T(W);
        [...]
    };
    [...]
}
```

-1- `iterator::iterator_conceptcategory` is defined as follows:

(1.1) — If `W` models *advanceable*, then `iterator_conceptcategory` is `random_access_iterator_tag`.

- (1.2) — Otherwise, if `w` models `decrementable`, then `iterator_category` is `bidirectional_iterator_tag`.
- (1.3) — Otherwise, if `w` models `incrementable`, then `iterator_category` is `forward_iterator_tag`.
- (1.4) — Otherwise, `iterator_category` is `input_iterator_tag`.
-

[3292^{\(i\)}](#). `iota_view` is under-constrained

Section: 24.6.3.2 [\[range.iota.view\]](#) **Status:** [Ready](#) **Submitter:** Barry Revzin **Opened:** 2019-09-13 **Last modified:** 2020-01-12

Priority: Not Prioritized

View all issues with [Ready](#) status.

Discussion:

[P1207R4](#) changed `weakly_incrementable` from requiring `semiregular` to requiring `default_constructible` && `movable`.

`iota_view` currently is specified to require that `w` be just `weakly_incrementable`. But we have to copy the `w` from the view into its iterator and also in `operator*()` to return a `w`.

The shortest resolution is just to add " && `semiregular<w>`" to the class constraints.

[Status to ready after discussion Friday morning in Belfast]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.6.3.2 [\[range.iota.view\]](#) as indicated:

```
namespace std::ranges {  
    [...]   
    template<weakly_incrementable W, semiregular Bound = unreachable_sentinel_t>  
        requires weakly_equality_comparable_with<W, Bound> && semiregular<W>  
        class iota_view : public view_interface<iota_view<W, Bound> {  
            [...]   
        };  
    [...]   
}
```

[3294^{\(i\)}](#). `zoned_time` deduction guides misinterprets `string/char*`

Section: 27.11.7.1 [\[time.zone.zonedtime.overview\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-09-14 **Last modified:** 2019-12-02

Priority: 0

View all other issues in [\[time.zone.zonedtime.overview\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The current deduction guide for `zoned_time` for the following declarations

```
zoned_time zpc("America/New_York", std::chrono::system_clock::now());  
zoned_time zps(std::string("America/New_York"), std::chrono::system_clock::now());
```

will attempt to produce a `zoned_time` instance with `const char*` (for `zpc`) and with `std::string` (for `zps`), respectively, as the deduced type for the `TimeZonePtr` template parameter. This is caused by the fact that the unconstrained `TimeZonePtr` deduction guide template will produce better candidates and will be selected by overload resolution.

The proposed resolution merges the deduction of the `std::string_view/TimeZonePtr` deduction guides into one guide, that deduces `const time_zone*` for any type convertible to `string_view`. This is necessary to override the deduction from `TimeZonePtr` constructor candidates.

In addition, we disable the deduction from `string_view` constructors, that would produce better candidates than the deduction guides and create `zoned_time` instances with durations coarser than seconds (causing similar issue as LWG [3232](#)):

```
std::chrono::local_time<hours> lh(10h);
std::chrono::zoned_time zt1("Europe/Helsinki", lh);
std::chrono::zoned_time zt2(std::string("Europe/Helsinki"), lh);
std::chrono::zoned_time zt3(std::string_view("Europe/Helsinki"), lh);
```

Without disabling the deduction from the `string_view` constructor, the type of the `zt3` variable would be deduced to `zoned_time<hours>`, with the proposed change the types of the variables `zt1`, `zt2`, and `zt3` are consistently deduced as `zoned_time<seconds>`.

Finally, the wording eliminates the unnecessary `zoned_time<Duration>` guide (covered by `zoned_time<Duration, TimeZonePtr2>`).

The change was implemented in the example implementation. The dedicated test can be found [here](#).

[2019-10-31 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 27.11.7.1 [[time.zone.zonedtime.overview](#)], class template `zoned_time` synopsis, as indicated:

```
namespace std::chrono {
    [...]
    zoned_time() -> zoned_time<seconds>;

    template<class Duration>
        zoned_time(sys_time<Duration>)
            -> zoned_time<common_type_t<Duration, seconds>>;

    template<class TimeZonePtrOrName>
    using time-zone-representation =
    conditional_t<is_convertible_v<TimeZonePtrOrName, string_view>,
    const time_zone*,
    remove_cv_ref<TimeZonePtrOrName>>; // exposition only

    template<class TimeZonePtrOrName>
    zoned_time(TimeZonePtrOrName&&)
    -> zoned_time<seconds, time-zone-representation<TimeZonePtr>>;

    template<class TimeZonePtrOrName, class Duration>
        zoned_time(TimeZonePtrOrName&&, sys_time<Duration>)
            -> zoned_time<common_type_t<Duration, seconds>,
                time-zone-representation<TimeZonePtrOrName>>;

    template<class TimeZonePtrOrName, class Duration>
        zoned_time(TimeZonePtrOrName&&, local_time<Duration>, choose = choose::earliest)
            -> zoned_time<common_type_t<Duration, seconds>,
                time-zone-representation<TimeZonePtrOrName>>;

    template<class TimeZonePtr, class Duration>
    zoned_time(TimeZonePtr, zoned_time<Duration>, choose = choose::earliest)
    -> zoned_time<common_type_t<Duration, seconds>, TimeZonePtr>;
    zoned_time(string_view) -> zoned_time<seconds>;

    template<class Duration>
    zoned_time(string_view, sys_time<Duration>)
    -> zoned_time<common_type_t<Duration, seconds>>;

    template<class Duration>
    zoned_time(string_view, local_time<Duration>, choose = choose::earliest)
    -> zoned_time<common_type_t<Duration, seconds>>;

    template<class Duration, class TimeZonePtrOrName, class TimeZonePtr2>
        zoned_time(TimeZonePtrOrName&&, zoned_time<Duration, TimeZonePtr2>, choose = choose::earliest)
            -> zoned_time<Duration, time-zone-representation<TimeZonePtrOrName>>;
}
```

-1- `zoned_time` represents a logical pairing of a `time_zone` and a `time_point` with precision `Duration`. `zoned_time<Duration>` maintains the invariant that it always refers to a valid time zone and represents a point in time that exists and is not ambiguous in that time zone.

-2- If `Duration` is not a specialization of `chrono::duration`, the program is ill-formed.

-?- Every constructor of `zoned_time` that accepts a `string_view` as first parameter does not participate in class template argument deduction (12.4.1.8 [over.match.class.deduct]).

[3296^{\(i\)}](#). Inconsistent default argument for `basic_regex<>::assign`

Section: 30.8 [\[re.regex\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Mark de Wever **Opened:** 2019-09-16 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in `[re.regex]`.

View all other [issues](#) in `[re.regex]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

The declaration of the overload of `basic_regex<>::assign(const charT* p, size_t len, flag_type f)` has an inconsistent default argument for the `flag_type f` parameter.

30.8 [\[re.regex\]](#) p3:

```
basic_regex& assign(const charT* p, size_t len, flag_type f);
```

30.8.2 [\[re.regex.assign\]](#) before p12:

```
basic_regex& assign(const charT* ptr, size_t len, flag_type f = regex_constants::ECMAScript);
```

Since all other overloads have a default argument in both 30.8 [\[re.regex\]](#) and 30.8.2 [\[re.regex.assign\]](#) I propose to add a default argument for this overload in the declaration in 30.8 [\[re.regex\]](#).

It should be pointed out that there exists implementation divergence due to the current wording state: `libc++` and `libstdc++` do *not* implement the default argument. The MS STL library *does* have the default argument.

[2019-10-31 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 30.8 [\[re.regex\]](#), class template `basic_regex` synopsis, as indicated:

```
[...]
// 30.8.2 \[re.regex.assign\], assign
[...]
basic_regex& assign(const charT* ptr, flag_type f = regex_constants::ECMAScript);
basic_regex& assign(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
template<class string_traits, class A>
    basic_regex& assign(const basic_string<charT, string_traits, A>& s,
                        flag_type f = regex_constants::ECMAScript);
template<class InputIterator>
    basic_regex& assign(InputIterator first, InputIterator last,
                        flag_type f = regex_constants::ECMAScript);
basic_regex& assign(initializer_list<charT>,
                    flag_type = regex_constants::ECMAScript);
[...]
```

[3299^{\(i\)}](#). Pointers don't need customized iterator behavior

Section: 23.3.3.1 [\[iterator.cust.move\]](#), 23.3.3.2 [\[iterator.cust.swap\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-10-07 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in `[iterator.cust.move]`.

View all other [issues](#) in `[iterator.cust.move]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

It is not intentional design that users may customize the behavior of `ranges::iter_move` (23.3.3.1 [\[iterator.cust.move\]](#)) and `ranges::iter_swap` (23.3.3.2 [\[iterator.cust.swap\]](#)) for pointers to program-defined type by defining e.g. `iter_move(my_type*)` or `iter_swap(my_type*, my_type*)` in a namespace associated with `my_type`. The intent of customization points is that users may define behavior for types they define, not that users may mess with the well-defined semantics for existing types like pointers.

We should forbid such silliness by constraining the "finds an overload via ADL" cases for customization points to only trigger with argument expressions of class or enumeration type. Note that WG21 made a similar change to `ranges::swap` shortly before merging it into the working draft to forbid users customizing behavior for pointers to program-defined types or arrays of program-defined types.

[2019-11-16 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4830](#).

[Drafting note: [3247](#) touches the same wording in 23.3.3.1 [\[iterator.cust.move\]](#); if both are resolved simultaneously the changes should be reconciled before passing them on to the Editor.]

1. Modify 23.3.3.1 [\[iterator.cust.move\]](#) as follows:

(1.1) — `iter_move(E)`, if ~~that expression is valid~~, **`E` has class or enumeration type and `iter_move(E)` is a well-formed expression** with overload resolution performed in a context that does not include a declaration of `ranges::iter_move`.

2. Modify 23.3.3.2 [\[iterator.cust.swap\]](#) as follows:

(4.1) — `(void)iter_swap(E1, E2)`, if ~~that expression is valid~~, **`either E1 or E2` has class or enumeration type and `iter_swap(E1, E2)` is a well-formed expression** with overload resolution performed in a context that includes the declaration

```
template<class I1, class I2>
void iter_swap(I1, I2) = delete;
```

and does not include a declaration of `ranges::iter_swap`. If the function selected by overload resolution does not exchange the values denoted by `E1` and `E2`, the program is ill-formed with no diagnostic required.

[3300^{\(i\)}](#). Non-array `ssize` overload is underconstrained

Section: 23.7 [\[iterator.range\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-09-27 **Last modified:** 2019-12-02

Priority: 3

View other [active issues](#) in [\[iterator.range\]](#).

View all other [issues](#) in [\[iterator.range\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The overload of `ssize` specified in 23.7 [\[iterator.range\]](#)/18 has no constraints, yet it specializes `make_signed_t` which has a precondition that its type parameter is an integral type or enumeration but not `bool` (20.15.7.3 [\[meta.trans.sign\]](#)). This precondition needs to be propagated to `ssize` as "*Mandates [or Constraints]: `decltype(c.size())` [meets the requirements for the type argument to `make_signed`]*". "*Mandates*" seems to be more in line with LWG guidance since there are no traits nor concepts that observe `ssize`.

[2019-11-16 Issue Prioritization]

Priority to 3 after reflector discussion.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

1. Modify 23.7 [\[iterator.range\]](#) as indicated:


```
template<class C> constexpr auto ssize(const C& c)
-> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
```

-?- Mandates: decltype(c.size()) is a (possibly cv-qualified) integral or enumeration type but not a bool type.

-18- Returns:

```
static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size())
```

[2019-10-28; Tim provides improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4835](#).

1. Modify 23.7 [\[iterator.range\]](#) as indicated:

```
template<class C> constexpr auto ssize(const C& c)
-> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
```

-?- Mandates: decltype(c.size()) is an integral or enumeration type other than bool.

-18- Returns:

```
static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size())
```

[2019-11-18; Casey comments and improves wording]

It would be better to provided the *Mandates*: guarantee in [tab:meta.trans.sign] instead of one special place where the make_signed template is used. The wording below attempts to realize that.

[2019-11-23 Issue Prioritization]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4835](#).

1. Change Table 52 — "Sign modifications" in [tab:meta.trans.sign] as indicated:

Table 52 — Sign modifications [tab:meta.trans.sign]

Template	Comments
template <class T> struct make_signed;	If T names a (possibly cv-qualified) signed integer type (6.8.1 [basic.fundamental]) then the member typedef type names the type T; otherwise, if T names a (possibly cv-qualified) unsigned integer type then type names the corresponding signed integer type, with the same cv-qualifiers as T; otherwise, type names the signed integer type with smallest rank (6.8.4 [conv.rank]) for which sizeof(T) == sizeof(type), with the same cv-qualifiers as T. Requires<u>Mandates</u>: T shall beis an (possibly cv-qualified) integral type or enumeration <u>type other than cv</u> but not a bool type.
template <class T> struct make_unsigned;	If T names a (possibly cv-qualified) unsigned integer type (6.8.1 [basic.fundamental]) then the member typedef type names the type T; otherwise, if T names a (possibly cv-qualified) signed integer type then type names the corresponding unsigned integer type, with the same cv-qualifiers as T; otherwise, type names the unsigned integer type with smallest rank (6.8.4 [conv.rank]) for which sizeof(T) == sizeof(type), with the same cv-qualifiers as T. Requires<u>Mandates</u>: T shall beis an (possibly cv-qualified) integral type or enumeration <u>type other than cv</u> but not a bool type.

2. Change 23.7 [\[iterator.range\]](#) as indicated:

```
template<class C> constexpr auto ssize(const C& c)
-> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
```

-18- ~~Returns~~Effects: Equivalent to:

```
return static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size()));
```

[3302\(i\)](#). Range adaptor objects keys and values are unspecified

Section: 24.2 [\[ranges.syn\]](#) **Status:** [Ready](#) **Submitter:** Michel Morin **Opened:** 2019-10-04 **Last modified:** 2019-12-02

Priority: 1

View other [active issues](#) in [\[ranges.syn\]](#).

View all other [issues](#) in [\[ranges.syn\]](#).

View all issues with [Ready](#) status.

Discussion:

This issue was submitted as editorial issue [cplusplus/draft#3231](#) but had been classified as non-editorial.

keys and values are listed in 24.2 [\[ranges.syn\]](#), but not specified. It seems that [P1035R7](#) forgot to specify them (as `elements<0>` and `elements<1>`).

[2019-10-31 Issue Prioritization]

Priority to 1 after reflector discussion.

[2019-11 Wednesday night issue processing in Belfast.]

Status to Ready.

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
namespace std::ranges {
    [...]

    template<class R>
        using keys_view = elements_view<all_view<R>, 0>;
    template<class R>
        using values_view = elements_view<all_view<R>, 1>;
    namespace views {
        template<size_t N>
            inline constexpr unspecified elements = unspecified;
            inline constexpr auto unspecified keys = elements<0>unspecified;
            inline constexpr auto unspecified values = elements<1>unspecified;
        }
    }
    [...]
}
```

[3303\(i\)](#). Bad "constexpr" marker for `destroy/destroy_n`

Section: 20.10.2 [\[memory.syn\]](#) **Status:** [Ready](#) **Submitter:** Jens Maurer **Opened:** 2019-10-10 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

This issue was submitted as editorial issue [cplusplus/draft#3181](#) but is considered non-editorial.

[P0784R7](#), approved in Cologne, added "constexpr" markers to the overloads of `destroy` and `destroy_n` taking an `ExecutionPolicy` parameter. This seems to be in error; parallel algorithms should not be marked "constexpr". (None of the parallel algorithms in `<algorithm>` is marked "constexpr".)

[2019-11 Marked as 'Ready' during Monday issue prioritization in Belfast]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 20.10.2 [\[memory.syn\]](#), header <memory> synopsis, as indicated:

```
namespace std {
    [...]

    // 20.10.11.9 \[specialized.destroy\], destroy
    template<class T>
        constexpr void destroy_at(T* location);
    template<class ForwardIterator>
        constexpr void destroy(ForwardIterator first, ForwardIterator last);
    template<class ExecutionPolicy, class ForwardIterator>
        constexpr void destroy(ExecutionPolicy&& exec, // see 25.3.5 \[algorithms.parallel.overloads\]
                               ForwardIterator first, ForwardIterator last);
    template<class ForwardIterator, class Size>
        constexpr ForwardIterator destroy_n(ForwardIterator first, Size n);
    template<class ExecutionPolicy, class ForwardIterator, class Size>
        constexpr ForwardIterator destroy_n(ExecutionPolicy&& exec, // see 25.3.5 \[algorithms.parallel.overloads\]
                                             ForwardIterator first, Size n);
    [...]
}
```

[3304](#)⁽ⁱ⁾. Allocate functions of `std::polymorphic_allocator` should require `[[nodiscard]]`

Section: 20.12.3 [\[mem.poly.allocator.class\]](#) **Status:** [Ready](#) **Submitter:** Hiroaki Ando **Opened:** 2019-10-16 **Last modified:** 2020-01-12

Priority: 3

View other [active issues](#) in `[mem.poly.allocator.class]`.

View all other [issues](#) in `[mem.poly.allocator.class]`.

View all issues with [Ready](#) status.

Discussion:

`[[nodiscard]]` is specified for `std::polymorphic_allocator<>::allocate()`.

But the allocate functions added with [P0339R6](#) doesn't have it.

Isn't `[[nodiscard]]` necessary for these functions?

[2019-11 Priority to 3 during Monday issue prioritization in Belfast]

[2019-11 After discussion with LEWG, assigning to LEWG]

[2019-11-4; Daniel comments]

This issue is related to LWG [3312](#).

[2019-11; Friday AM in Belfast. Status changed to "Ready"]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 20.12.3 [\[mem.poly.allocator.class\]](#), class template `polymorphic_allocator` synopsis, as indicated:

```
namespace std::pmr {
    template<class Tp = byte> class polymorphic_allocator {
        [...]
        // 20.12.3.2 \[mem.poly.allocator.mem\], member functions
        [[nodiscard]] Tp* allocate(size_t n);
        void deallocate(Tp* p, size_t n);

        [[nodiscard]] void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
        void deallocate_bytes(void* p, size_t nbytes, size_t alignment = alignof(max_align_t));
        template<class T> [[nodiscard]] T* allocate_object(size_t n = 1);
        template<class T> void deallocate_object(T* p, size_t n = 1);
        template<class T, class... CtorArgs> [[nodiscard]] T* new_object(CtorArgs&&... ctor_args);
        template<class T> void delete_object(T* p);
        [...]
    };
}
```

2. Modify 20.12.3.2 [\[mem.poly allocator.mem\]](#) as indicated:

```
[nodiscard] void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
```

-5- *Effects*: Equivalent to: `return memory_rsrc->allocate(nbytes, alignment);`

[...]

[...]

```
template<class T>
```

```
[nodiscard] T* allocate_object(size_t n = 1);
```

-8- *Effects*: Allocates memory suitable for holding an array of n objects of type T , as follows:

(8.1) — if `SIZE_MAX / sizeof(T) < n`, throws `length_error`,

(8.2) — otherwise equivalent to:

```
return static_cast<T*>(allocate_bytes(n*sizeof(T), alignof(T)));
```

[...]

```
template<class T, class CtorArgs...>
```

```
[nodiscard] T* new_object(CtorArgs&&... ctor_args);
```

-11- *Effects*: Allocates and constructs an object of type T , as follows. Equivalent to:

```
T* p = allocate_object<T>();
try {
    construct(p, std::forward<CtorArgs>(ctor_args)...);
} catch (...) {
    deallocate_object(p);
    throw;
}
return p;
```

[...]

[3307\(i\)](#). `std::allocator<void>().allocate(n)`

Section: 20.10.10 [\[default.allocator\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-10-25 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [\[default.allocator\]](#).

View all other [issues](#) in [\[default.allocator\]](#).

View all issues with [Ready](#) status.

Discussion:

In C++20 the `std::allocator<void>` explicit specialization is gone, which means it uses the primary template, which has `allocate` and `deallocate` members.

Although it's implied by the use of `sizeof(T)`, `std::allocator<T>::allocate` doesn't have an explicit precondition that the value type is complete.

[2019-11 Status to 'Ready' in Monday issue prioritization in Belfast]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 20.10.10.1 [\[allocator.members\]](#) as indicated:

```
[nodiscard] constexpr T* allocate(size_t n);
```

```
[-? - Mandates:  $T$  is not an incomplete type (6.8 \[basic.types\]).
```

-2- *Returns*: A pointer to the initial element of an array of storage of size $n * \text{sizeof}(T)$, aligned appropriately for objects of type T .

[...]

[3310^{\(i\)}](#). Replace `SIZE_MAX` with `numeric_limits<size_t>::max()`

Section: 20.12.3.2 [[mem.poly.allocator.mem](#)] **Status:** [Ready](#) **Submitter:** Japan **Opened:** 2019-11-04 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [[mem.poly.allocator.mem](#)].

View all other [issues](#) in [[mem.poly.allocator.mem](#)].

View all issues with [Ready](#) status.

Discussion:

Addresses JP 218/219

It's better to use a C++ property than C standard library macro, `SIZE_MAX`.

[2019-11 Status to Ready during Tuesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 20.12.3.2 [[mem.poly.allocator.mem](#)] as indicated:

```
[[nodiscard]] Tp* allocate(size_t n);
```

-1- *Effects:* If `SIZE_MAX`~~`numeric_limits<size_t>::max()`~~ / `sizeof(Tp)` < `n`, throws `length_error`. [...]

[...]

```
template<class T>
```

```
    T* allocate_object(size_t n = 1);
```

-8- *Effects:* Allocates memory suitable for holding an array of `n` objects of type `T`, as follows:

(8.1) — if `SIZE_MAX`~~`numeric_limits<size_t>::max()`~~ / `sizeof(T)` < `n`, throws `length_error`,

(8.2) — otherwise equivalent to:

```
    return static_cast<T*>(allocate_bytes(n*sizeof(T), alignof(T)));
```

[3313^{\(i\)}](#). `join_view::iterator::operator--` is incorrectly constrained

Section: 24.7.10.3 [[range.join.iterator](#)] **Status:** [Ready](#) **Submitter:** United States **Opened:** 2019-11-04 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [[range.join.iterator](#)].

View all other [issues](#) in [[range.join.iterator](#)].

View all issues with [Ready](#) status.

Discussion:

Addresses US 294

`join_view::iterator::operator--` is improperly constrained. In the *Effects:* clause in paragraph 14, we see the statement:

```
    inner_ = ranges::end(*--outer_);
```

However, this only well-formed when `end` returns an iterator, not a sentinel. This requirement is not reflected in the constraints of the function(s).

Eric Niebler:

From the WD, `join_view::iterator::operator--` is specified as:

```
constexpr iterator& operator--()
    requires ref_is_glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>>;
```

-14- *Effects*: Equivalent to:

```
if (outer_ == ranges::end(parent_>base_))
    inner_ = ranges::end(*--outer_);
while (inner_ == ranges::begin(*outer_))
    inner_ = ranges::end(*--outer_);
--inner_;
return *this;
```

The trouble is from the lines that do:

```
inner_ = ranges::end(*--outer_);
```

Clearly this will only compile when `*--outer` returns a `common_range`, but nowhere is that requirement stated.

[2019-11 Status to Ready during Tuesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 24.7.10.3 [\[range.join.iterator\]](#), class template `join_view::iterator` synopsis, as indicated:

```
constexpr iterator& operator--()
    requires ref_is_glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;

constexpr iterator operator--(int)
    requires ref_is_glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;
```

2. Modify 24.7.10.3 [\[range.join.iterator\]](#) as indicated:

```
constexpr iterator& operator--()
    requires ref_is_glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;
```

-14- *Effects*: Equivalent to:

```
if (outer_ == ranges::end(parent_>base_))
    inner_ = ranges::end(*--outer_);
while (inner_ == ranges::begin(*outer_))
    inner_ = ranges::end(*--outer_);
--inner_;
return *this;

constexpr iterator operator--(int)
    requires ref_is_glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;
```

-15- *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

[3315^{\(i\)}](#). Correct Allocator Default Behavior

Section: 16.5.3.5 [\[allocator.requirements\]](#) **Status:** [Ready](#) **Submitter:** United States **Opened:** 2019-11-04 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [\[allocator.requirements\]](#).

View all other [issues](#) in [\[allocator.requirements\]](#).

View all issues with [Ready](#) status.

Discussion:

Addresses [US 162/US 163](#)

[US 162:](#)

The default behavior for `a.destroy` is now to call `destroy_at`

Proposed change:

Replace "default" entry with: `destroy_at(c)`

[US 163:](#)

The default behavior for `a.construct` is now to call `construct_at`

Proposed change:

Replace "default" entry with: `construct_at(c, std::forward<Args>(args)...)`

Dietmar Kühl:

In Table 34 [tab:cpp17 allocator] the behavior of `a.construct(c, args)` and `a.destroy(c)` are described to have a default behavior of `::new ((void*)c) C(forward<Args>(args))` and `c->~C()`, respectively. However, this table doesn't actually define what is happening if these operations are omitted: The behavior is provided when using an allocator is used via `std::allocator_traits` and is, thus, defined by the corresponding `std::allocator_traits` functions. These functions are specified in 20.10.9.2 [\[allocator.traits.members\]](#) paragraphs 5 and 6 to call `construct_at(c, std::forward<Args>(args))` and `destroy_at(p)`, respectively. The text in the table should be updated to match the actual behavior.

[2019-11 Status to Ready during Wednesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

- 1. Modify 16.5.3.5 [\[allocator.requirements\]](#), Table [tab:cpp17.allocator] "*Cpp17Allocator requirements*" as indicated:

Table 34 — *Cpp17Allocator requirements* [tab:cpp17.allocator]

Expression	Return type	Assertion/note pre-/post-condition	Default
...			
<code>a.construct(c, args)</code>	(not used)	<i>Effects:</i> Constructs an object of type <code>C</code> at <code>c</code>	<code>::new ((void*)c) C</code> <code>(construct_at(c, std::forward<Args>(args)...))</code>
<code>a.destroy(c)</code>	(not used)	<i>Effects:</i> Destroys the object at <code>c</code>	<code>c->~C()</code> <code>destroy_at(c)</code>
...			

[3316^{\(i\)}](#). Correctly define epoch for `utc_clock` / `utc_timepoint`

Section: 27.7.2.1 [\[time.clock.utc.overview\]](#) Status: [Ready](#) Submitter: Great Britain Opened: 2019-11-05 Last modified: 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [GB 333](#)

UTC epoch is not correctly defined UTC has an officially recorded epoch of 1/1/1972 00:00:00 and is 10 seconds behind TAI. This can be confirmed through reference to the [BIPM](#) (the body that oversees international metrology)

"The defining epoch of 1 January 1972, 0 h 0m 0 s UTC was set 10 s behind TAI, which was the approximate accumulated difference between TAI and UT1 since the inception of TAI in 1958, and a unique fraction of a second adjustment was applied so that UTC would differ from TAI by an integral number of seconds. The recommended maximum departure of UTC from UT1 was 0.7 s. The term "leap second" was introduced for the stepped second."

Proposed change:

utc_clock and utc_timepoint should correctly report relative to the official UTC epoch. 27.2.2.1 footnote 1 should read:

In contrast to sys_time, which does not take leap seconds into account, utc_clock and its associated time_point, utc_time, count time, including leap seconds, since 1972-01-01 00:00:00 UTC. [Example: clock_cast<utc_clock>(sys_seconds{sys_days{1972y/January/1}}).time_since_epoch() is 0s. clock_cast<utc_clock>(sys_seconds{sys_days{2000y/January/1}}).time_since_epoch() is 883'612'822, which is 10'197 * 86'400s + 22s. — end example]

Howard Hinnant:

Clarify that the epoch of utc_clock is intended to be 1970-01-01.

Rationale: The main use case of utc_clock is to get the correct number of seconds when subtracting time points straddling a leap second insertion point, and this computation is independent of the epoch. Furthermore learning/teaching that utc_clock is system_clock except that utc_clock includes leap seconds is easier. And this fact is more easily understood when comparing the underlying .time_since_epoch() of equivalent time points from each clock.

[2019-11 Status to Ready during Wednesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 27.7.2.1 [\[time.clock.utc.overview\]](#) as indicated:

-1- In contrast to sys_time, which does not take leap seconds into account, utc_clock and its associated time_point, utc_time, count time, including leap seconds, since 1970-01-01 00:00:00 UTC. [Note: The UTC time standard began on 1972-01-01 00:00:10 TAI. To measure time since this epoch instead, one can add/subtract the constant sys_days{1972y/1/1} - sys_days{1970y/1/1} (63'072'000s) from the utc time — end note] [Example: clock_cast<utc_clock>(sys_seconds{sys_days{1970y/January/1}}).time_since_epoch() is 0s. clock_cast<utc_clock>(sys_seconds{sys_days{2000y/January/1}}).time_since_epoch() is 946'684'822s, which is 10'957 * 86'400s + 22s. — end example]

[3317^{\(i\)}](#). Incorrect operator<< for floating-point durations

Section: 27.5.10 [\[time.duration.io\]](#) Status: [Ready](#) Submitter: United States Opened: 2019-11-05 Last modified: 2019-12-02

Priority: 0

View other [active issues](#) in [time.duration.io].

View all other [issues](#) in [time.duration.io].

View all issues with [Ready](#) status.

Discussion:

Addresses [US 334](#)

operator<< for floating-point durations always produces output with six digits after the decimal point, and doesn't use the stream's locale either.

Proposed change:

Rewrite the specification to not rely on to_string() for floating-point formatting.

[2019-11 Status to Ready during Wednesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 27.5.10 [\[time.duration.io\]](#) as indicated:

```
template<class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

~~-1- Requires: Rep is an integral type whose integer conversion rank (6.8.4 [\[conv.rank\]](#)) is greater than or equal to that of short, or a floating-point type. charT is char or wchar_t.~~

-2- **Effects:** Forms a `basic_string<charT, traits>` from `d.count()` using `to_string` if `charT` is `char`, or `to_wstring` if `charT` is `wchar_t`. Appends the units suffix described below to the `basic_string`. Inserts the resulting `basic_string` into `os`. [Note: This specification ensures that the result of this streaming operation will obey the width and alignment properties of the stream. — end note] Inserts the duration `d` onto the stream `os` as if it were implemented as follows:

```
basic_ostringstream<charT, traits> s;  
s.flags(os.flags());  
s.imbue(os.getloc());  
s.precision(os.precision());  
s << d.count() << units_suffix;  
return os << s.str();
```

-3- The units suffix `units_suffix` depends on the type `Period::type` as follows:

- (3.1) — If `Period::type` is `atto`, the suffix `units_suffix` is "as".
- (3.2) — Otherwise, if `Period::type` is `femto`, the suffix `units_suffix` is "fs".
- (3.3) — Otherwise, if `Period::type` is `pico`, the suffix `units_suffix` is "ps".
- (3.4) — Otherwise, if `Period::type` is `nano`, the suffix `units_suffix` is "ns".
- (3.5) — Otherwise, if `Period::type` is `micro`, the suffix `units_suffix` is "µs" ("u00b5u0073").
- (3.6) — Otherwise, if `Period::type` is `milli`, the suffix `units_suffix` is "ms".
- (3.7) — Otherwise, if `Period::type` is `centi`, the suffix `units_suffix` is "cs".
- (3.8) — Otherwise, if `Period::type` is `deci`, the suffix `units_suffix` is "ds".
- (3.9) — Otherwise, if `Period::type` is `ratio<1>`, the suffix `units_suffix` is "s".
- (3.10) — Otherwise, if `Period::type` is `deca`, the suffix `units_suffix` is "das".
- (3.11) — Otherwise, if `Period::type` is `hecto`, the suffix `units_suffix` is "hs".
- (3.12) — Otherwise, if `Period::type` is `kilo`, the suffix `units_suffix` is "ks".
- (3.13) — Otherwise, if `Period::type` is `mega`, the suffix `units_suffix` is "Ms".
- (3.14) — Otherwise, if `Period::type` is `giga`, the suffix `units_suffix` is "Gs".
- (3.15) — Otherwise, if `Period::type` is `tera`, the suffix `units_suffix` is "Ts".
- (3.16) — Otherwise, if `Period::type` is `peta`, the suffix `units_suffix` is "Ps".
- (3.17) — Otherwise, if `Period::type` is `exa`, the suffix `units_suffix` is "Es".
- (3.18) — Otherwise, if `Period::type` is `ratio<60>`, the suffix `units_suffix` is "min".
- (3.19) — Otherwise, if `Period::type` is `ratio<3600>`, the suffix `units_suffix` is "h".
- (3.20) — Otherwise, if `Period::type` is `ratio<86400>`, the suffix `units_suffix` is "d".
- (3.21) — Otherwise, if `Period::type::den == 1`, the suffix `units_suffix` is "[num]s".
- (3.22) — Otherwise, the suffix `units_suffix` is "[num/den]s".

In the list above the use of `num` and `den` refer to the static data members of `Period::type`, which are converted to arrays of `charT` using a decimal conversion with no leading zeroes.

-4- If `Period::type` is `micro`, but the character `U+00B5` cannot be represented in the encoding used for `charT`, the unit suffix `units_suffix` "us" is used instead of "µs".

-5- **Returns:** `os`.

3318⁽ⁱ⁾. Clarify whether clocks can represent time before their epoch

Section: 27.7.1.1 [[time.clock.system.overview](#)] **Status:** [Ready](#) **Submitter:** Great Britain **Opened:** 2019-11-05 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [GB 335](#)

Wording for clocks should be unified unless they are intended to behave differently In 27.7.1.1 note 1 for `system_clock` it is stated:

"Objects of type `system_clock` represent wall clock time from the system-wide realtime clock. Objects of type `sys_time<Duration>` measure time since (and before) 1970-01-01 00:00:00 UTC"

The express statement of "since (and before)" is important given the time epoch of these clocks. If all the clocks support time prior to their zero-time then this should be stated explicitly. If not then likewise that should be noted. No change is proposed yet, clarification required over the intended behaviour when using values prior to a given clock's epoch is needed before the appropriate change can be suggested.

Proposed change:

Unify the wording.

Howard Hinnant:

The clocks that are specified to have a signed rep imply that they will support negative time points, but not *how* negative. For example if `system_clock::duration` is nanoseconds represented with 64 bits, then `system_clock::time_point` can't possibly represent dates prior to 1677-09-21 00:12:43.145224192. This is a negative `time_point` since it is prior to 1970-01-01 00:00:00. But it is not very negative compared to (for example) `sys_time<microseconds>::min()`.

Those clocks with a signed rep are:

- `system_clock`
- `utc_clock`
- `tai_clock`
- `gps_clock`
- `file_clock`

Those clocks where the signed-ness of rep is unspecified are:

- `steady_clock`
- `high_resolution_clock`

Therefore this response emphasizes the "Unify the wording" part of this NB comment.

[2019-11 Status to Ready during Wednesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 27.7.1.1 [\[time.clock.system.overview\]](#) as indicated:

-1- Objects of type `system_clock` represent wall clock time from the system-wide realtime clock. Objects of type `sys_time<Duration>` measure time since (and before) 1970-01-01 00:00:00 UTC excluding leap seconds. This measure is commonly referred to as Unix time. This measure facilitates an efficient mapping between `sys_time` and calendar types (27.8 [\[time.cal\]](#)). *[Example: `sys_seconds{sys_days{1970y/January/1}}`.time_since_epoch() is 0s. `sys_seconds{sys_days{2000y/January/1}}`.time_since_epoch() is 946'684'800s, which is 10'957 * 86'400s. — end example]*

[3319](#)⁽ⁱ⁾. Properly reference specification of IANA time zone database

Section: 27.11.1 [\[time.zone.general\]](#) **Status:** [Ready](#) **Submitter:** Germany **Opened:** 2019-11-05 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [DE 344](#)

This paragraph says

"27.11 describes an interface for accessing the IANA Time Zone database described in RFC 6557, ..."

However, RFC 6557 does not describe the database itself; it only describes the maintenance procedures for that database, as its title implies (quoted in clause 2).

Proposed change:

Add a reference to a specification of the database itself, or excise all references to the IANA time zone database.

Howard Hinnant:

We can not entirely remove the reference to IANA because we need portable `time_zone` names (e.g. "America/New_York") and definitions. However the NB comment is quite accurate and fixed with the proposed resolution.

[2019-11 Status to Ready during Wednesday morning issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 2 [\[intro.refs\]](#) as indicated:

-1- The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

[...]

~~(1.2) — INTERNET ENGINEERING TASK FORCE (IETF). RFC 6557: Procedures for Maintaining the Time Zone Database [online]. Edited by E. Lear, P. Eggert. February 2012 [viewed 2018-03-26]. Available at <https://www.ietf.org/rfc/rfc6557.txt>~~

[...]

2. Modify 27.11.1 [\[time.zone.general\]](#) as indicated:

-1- 27.11 [\[time.zone\]](#) describes an interface for accessing the IANA Time Zone ~~d~~Database ~~described in RFC 6557~~, that interoperates with `sys_time` and `local_time`. This interface provides time zone support to both the civil calendar types (27.8 [\[time.cal\]](#)) and to user-defined calendars.

3. Modify section "Bibliography" as indicated:

The following documents are cited informatively in this document.

~~— IANA Time Zone Database. Available at <https://www.iana.org/time-zones>~~

— ISO/IEC 10967-1:2012, *Information technology — Language independent arithmetic — Part 1: Integer and floating point arithmetic*

[...]

[3320^{\(i\)}](#). `span::cbegin/cend` methods produce different results than `std::[ranges::]cbegin/cend`

Section: 22.7.3.7 [\[span.iterators\]](#) Status: [Ready](#) Submitter: Poland Opened: 2019-11-06 Last modified: 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [PL 247](#)

`span<T>` provides a const-qualified `begin()` method and `cbegin()` method that produces a different result if `T` is not const-qualified:

1. `begin()` produces mutable iterator over `T` (as if `T*`)
2. `cbegin()` produces const iterator over `T` (as if `T const*`)

As consequence for the object `s` of type `span<T>`, the call to the `std::cbegin(s)/std::ranges::cbegin(s)` produces different result than `s.cbegin()`.

Proposed change:

Change `span<T>` members `cbegin()/cend()/crbegin()/crend()/const_iterator` to be equivalent to `begin()/end()/rbegin()/rend()/iterator` respectively.

Tomasz Kamiński:

Per [LEWG discussion in Belfast](#) these methods and related typedefs should be removed.

[2019-11 Status to Ready during Wednesday night issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 22.7.3.1 [\[span.overview\]](#), class template `span` synopsis, as indicated:

```
namespace std {
    template<class ElementType, size_t Extent = dynamic_extent>
    class span {
    public:
        // constants and types
        using element_type = ElementType;
        using value_type = remove_cv_t<ElementType>;
        using index_type = size_t;
        using difference_type = ptrdiff_t;
        using pointer = element_type*;
        using const_pointer = const element_type*;
        using reference = element_type&;
        using const_reference = const element_type&;
        using iterator = implementation-defined; // see 22.7.3.7 \[span.iterators\]
        using const_iterator = implementation-defined;
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;
        static constexpr index_type extent = Extent;

        [...]
        // 22.7.3.7 \[span.iterators\], iterator support
        constexpr iterator begin() const noexcept;
        constexpr iterator end() const noexcept;
        constexpr const_iterator cbegin() const noexcept;
        constexpr const_iterator cend() const noexcept;
        constexpr reverse_iterator rbegin() const noexcept;
        constexpr reverse_iterator rend() const noexcept;
        constexpr const_reverse_iterator crbegin() const noexcept;
        constexpr const_reverse_iterator crend() const noexcept;
        friend constexpr iterator begin(span s) noexcept { return s.begin(); }
        friend constexpr iterator end(span s) noexcept { return s.end(); }
        [...]
    };
    [...]
}
```

2. Modify 22.7.3.7 [\[span.iterators\]](#) as indicated:

```
using iterator = implementation-defined;
using const_iterator = implementation-defined;
```

-1- The types `models` `contiguous_iterator` (23.3.4.14 [\[iterator.concept.contiguous\]](#)), meets the `Cpp17RandomAccessIterator` requirements (23.3.5.6 [\[random.access.iterators\]](#)), and meets the requirements for `constexpr` iterators (23.3.1 [\[iterator.requirements.general\]](#)). All requirements on container iterators (22.2 [\[container.requirements\]](#)) apply to `span::iterator` ~~and `span::const_iterator`~~ as well.

[...]

```
constexpr const_iterator cbegin() const noexcept;
```

~~-6- Returns: A constant iterator referring to the first element in the span. If `empty()` is true, then it returns the same value as `cend()`.~~

~~constexpr const_iterator cend() const noexcept;~~

~~-7- Returns: A constant iterator which is the past-the-end value.~~

~~constexpr const_reverse_iterator rbegin() const noexcept;~~

~~-8- Effects: Equivalent to: return const_reverse_iterator(cend());~~

~~constexpr const_reverse_iterator rend() const noexcept;~~

~~-9- Effects: Equivalent to: return const_reverse_iterator(rbegin());~~

[3321^{\(i\)}](#). uninitialized_construct_using_allocator should use construct_at

Section: 20.10.8.2 [\[allocator.uses.construction\]](#) **Status:** [Ready](#) **Submitter:** United States **Opened:** 2019-11-06 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [\[allocator.uses.construction\]](#).

View all other [issues](#) in [\[allocator.uses.construction\]](#).

View all issues with [Ready](#) status.

Discussion:

Addresses [US 213](#)

uninitialized_construct_using_allocator should use construct_at instead of operator new

Proposed change:

Effects: Equivalent to:

```
return ::new(static_cast<void*>(p))
construct_at(p,
T(make_obj_using_allocator<T>(alloc,
std::forward<Args>(args)...)));
```

Tim Song:

The proposed wording in the NB comment is incorrect, because it prevents guaranteed elision.

[2019-11 Status to Ready during Wednesday night issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 20.10.8.2 [\[allocator.uses.construction\]](#) as indicated:

```
template<class T, class Alloc, class... Args>
constexpr T* uninitialized_construct_using_allocator(T* p, const Alloc& alloc, Args&&... args);
```

~~-17- Effects:~~ Equivalent to:

```
return ::new(static_cast<void*>(p))
T(make_obj_using_allocator<T>(apply([&](auto&&...xs) {
    return construct_at(p, std::forward<decltype(xs)>(xs)...);
}), uses_allocator_construction args<T>(alloc, std::forward<Args>(args)...));
```

[3323^{\(i\)}](#). has-tuple-element helper concept needs convertible_to

Section: 24.7.15.2 [\[range.elements.view\]](#) **Status:** [Ready](#) **Submitter:** Great Britain **Opened:** 2019-11-06 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [GB 299](#)

has-tuple-element helper concept needs convertible_to

The exposition-only *has-tuple-element* concept (for `elements_view`) is defined as

```
template<class T, size_t N>
concept has-tuple-element = exposition only
requires(T t) {
    typename tuple_size<T>::type;
    requires N < tuple_size_v<T>;
    typename tuple_element_t<N, T>;
    { get<N>(t) } -> const tuple_element_t<N, T>&;
};
```

However, the return type constraint for `{ get<N>(t) }` is no longer valid under the latest concepts changes

Proposed change:

Change to:

```
template<class T, size_t N>
concept has-tuple-element = exposition only
requires(T t) {
    typename tuple_size<T>::type;
    requires N < tuple_size_v<T>;
    typename tuple_element_t<N, T>;
    { get<N>(t) } -> convertible_to<const tuple_element_t<N, T>&>;
};
```

Jonathan Wakely:

The NB comment says "The return type constraint for `{ get(t) }` is no longer valid under the latest concepts changes." The changes referred to are those in [P1452R2](#).

[2019-11 Status to Ready during Wednesday night issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 24.7.15.2 [\[range.elements.view\]](#), class template `elements_view` synopsis, as indicated:

```
namespace std::ranges {
    template<class T, size_t N>
    concept has-tuple-element = // exposition only
    requires(T t) {
        typename tuple_size<T>::type;
        requires N < tuple_size_v<T>;
        typename tuple_element_t<N, T>;
        { get<N>(t) } -> convertible_to<const tuple_element_t<N, T>&>;
    };
    [...]
}
```

[3324^{\(i\)}](#). Special-case `std::strong/weak/partial_order` for pointers

Section: 17.11.7 [\[cmp.alg\]](#) **Status:** [Ready](#) **Submitter:** Canada **Opened:** 2019-11-06 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [CA 178](#)

`std::strong_order`, `weak_order`, and `partial_order` have special cases for floating point, but are missing special casing for pointers. `compare_three_way` and `std::less` have the special casing for pointers.

Proposed change:

Change [\[cmp.alg\]](#) bullet 1.4 from

"Otherwise, `strong_ordering(E <=> F)` if it is a well-formed expression."

to
"Otherwise, strong_ordering(compare_three_way()(E, F)) if it is a well-formed expression."

Change [cmp.alg] bullet 2.4 from
"Otherwise, weak_ordering(E <=> F) if it is a well-formed expression."

to
"Otherwise, weak_ordering(compare_three_way()(E, F)) if it is a well-formed expression."

Change [cmp.alg] bullet 3.3 from
"Otherwise, partial_ordering(E <=> F) if it is a well-formed expression."

to
"Otherwise, partial_ordering(compare_three_way()(E, F)) if it is a well-formed expression."

Dietmar Kühl:

Use compare_three_way instead of <=> for the various comparison algorithms.

[2019-11 Status to Ready during Wednesday night issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Change 17.11.7 [cmp.alg] as indicated:

-1- The name strong_order [...]

[...]

(1.4) — Otherwise, strong_ordering(~~E <=> F~~compare_three_way()(E, F)) if it is a well-formed expression.

[...]

-2- The name weak_order [...]

[...]

(2.4) — Otherwise, weak_ordering(~~E <=> F~~compare_three_way()(E, F)) if it is a well-formed expression.

[...]

-3- The name partial_order [...]

[...]

(3.3) — Otherwise, partial_ordering(~~E <=> F~~compare_three_way()(E, F)) if it is a well-formed expression.

[...]

[3325^{\(i\)}](#). Constrain return type of transformation function for transform_view

Section: 24.7.5.2 [[range.transform.view](#)] **Status:** [Ready](#) **Submitter:** United States **Opened:** 2019-11-06 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [US 303](#)

The transform_view does not constrain the return type of the transformation function. It is invalid to pass a void-returning transformation function to the transform_view, which would cause its iterators' operator* member to return void.

Proposed change:

Change the constraints on transform_view to the following:

```
template<input_range V, copy_constructible F>
    requires view<V> && is_object_v<F> &&
        regular_invocable<F&, range_reference_t<V>> &&
        can-reference<invoke_result_t<F&, range_reference_t<V>>>
class transform_view;
```

Jonathan Wakely:

The NB comment says "The transform_view does not constrain the return type of the transformation function. It is invalid to pass a void-returning transformation function to the transform_view, which would cause its iterators' operator* member to return void."

[2019-11 Status to Ready during Wednesday night issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 24.7.5.2 [\[range.transform.view\]](#), class template elements_view synopsis, as indicated:

```
namespace std::ranges {
    template<input_range V, copy_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
            can-reference<invoke result t<F&, range reference t<V>>>
    class transform_view : public view_interface<transform_view<V, F>> {
    [...]
    };
    [...]
}
```

[3326^{\(i\)}](#). enable_view has false positives

Section: 24.4.4 [\[range.view\]](#) **Status:** [Ready](#) **Submitter:** Germany **Opened:** 2019-11-06 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [DE 282](#)

"Since the difference between range and view is largely semantic, the two are differentiated with the help of enable_view." (§3)

enable_view is designed as an opt-in trait to specify that a type is a view. It defaults to true for types derived from view_base (§4.2) which is clearly a form of opt-in. But it also employs a heuristic assuming that anything with iterator == const_iterator is also view (§4.3).

This is a very poor heuristic, the same paragraph already needs to define six exceptions from this rule for standard library types (§4.2).

Experience in working with range-v3 has revealed multiple of our own library types as being affected from needing to opt-out from the "auto-opt-in", as well. This is counter-intuitive: something that was never designed to be a view shouldn't go through hoops so that it isn't treated as a view.

Proposed change:

Make enable_view truly be opt-in by relying only on explicit specialisation or inheritance from view_base. This means removing 24.4.4 §4.2 - §4.4 and introducing new §4.2 "Otherwise, false".

Double-check if existing standard library types like basic_string_view and span need to opt-in to being a view now.

Casey Carter:

enable_view (24.4.4 [\[range.view\]](#)) is designed as an opt-in trait to specify that a type is a view. It defaults to true for types derived from view_base — which is a form of opt-in — and it also employs a heuristic. Unfortunately, the heuristic has false positives. The working draft itself includes six exceptions to the heuristic for standard library types. Since false positives are much more problematic for users than false negatives, we should eliminate the heuristic.

[2019-11 Status to Ready during Wednesday night issue processing in Belfast.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 24.4.4 [\[range.view\]](#) as indicated:

```
template<class T>
    inline constexpr bool enable_view = see below derived_from<T, view_base>;
```

~~4 Remarks: For a type T, the default value of enable_view<T> is:~~

~~(4.1) If derived_from<T, view_base> is true, true.~~

~~(4.2) Otherwise, if T is a specialization of class template initializer_list (17.10 [\[support.initlist\]](#)), set (22.4.6 [\[set\]](#)), multiset (22.4.7 [\[multiset\]](#)), unordered_set (22.5.6 [\[unord.set\]](#)), unordered_multiset (22.5.7 [\[unord.multiset\]](#)), or match_results (30.10 [\[re.results\]](#)), false.~~

~~(4.3) Otherwise, if both T and const T model range and range_reference_t<T> is not the same type as range_reference_t<const T>, false. [Note: Deep const-ness implies element ownership, whereas shallow const-ness implies reference semantics. — end note]~~

~~(4.4) Otherwise, true.~~

2. Modify 21.4.1 [\[string.view.synop\]](#), header <string_view> synopsis, as indicated:

```
namespace std {
    // 21.4.2 \[string.view.template\], class template basic_string_view
    template<class charT, class traits = char_traits<charT>>
        class basic_string_view;

    template<class charT, class traits>
    inline constexpr bool ranges::enable_view<basic_string_view<charT, traits>> = true;

    [...]
}
```

3. Modify 22.7.2 [\[span.syn\]](#), header <apsn> synopsis, as indicated:

```
namespace std {
    // constants
    inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();

    // 22.7.3 \[views.span\], class template span
    template<class ElementType, size_t Extent = dynamic_extent>
        class span;

    template<class ElementType, size_t Extent>
    inline constexpr bool ranges::enable_view<span<ElementType, Extent>> = Extent == 0 ||
    Extent == dynamic_extent;

    [...]
}
```

4. Modify 24.7.11.4 [\[range.split.outer.value\]](#), class split_view::outer_iterator::value_type synopsis, as indicated:

[Drafting note: The following applies the proposed wording for LWG [3276](#)]

```
namespace std::ranges {
    template<class V, class Pattern>
    template<bool Const>
    struct split_view<V, Pattern>::outer_iterator<Const>::value_type
        : view_interface<value_type> {
    private:
        outer_iterator i_ = outer_iterator(); // exposition only
    public:
        value_type() = default;
        constexpr explicit value_type(outer_iterator i);

        constexpr inner_iterator<Const> begin() const;
        constexpr default_sentinel_t end() const;
    };
}
```

[3327^{\(i\)}](#). Format alignment specifiers vs. text direction

Section: 20.20.2.2 [\[format.string.std\]](#) Status: [Ready](#) Submitter: Great Britain Opened: 2019-11-07 Last modified: 2019-12-02

Priority: 0

View other [active issues](#) in [format.string.std].

View all other [issues](#) in [format.string.std].

View all issues with [Ready](#) status.

Discussion:

Addresses [GB 225](#)

std::format() alignment specifiers should be independent of text direction The align specifiers for formatting standard integer and string types are expressed in terms of "left" and "right". However, "left alignment" as currently defined in the format() specification might end up being right-aligned when the resulting string is displayed in a RTL or bidirectional locale. This ambiguity can be resolved by removing "left" and "right" and replacing with "start" and "end", without changing any existing implementation and without changing the intent of the feature.

Proposed change:

In [tab:format.align]: Forces the field to be ~~left-aligned within~~ **aligned to the start of** the available space and Forces the field to be ~~right-aligned within~~ **aligned to the end of** the available space

Jeff Garland:

[Wiki notes from Belfast Wed:](#)

GB225

JG: SG16 approved this.

JG: If you scroll back up, you'll see see it's very tiny. Two line change.

JG: I'm willing to submit an LWG issue to suggest we make a wording change to take it off our plate.

CC: Is this the one that changes left/right to beginning/end?

Some people: yes

MC: Any problem with Jeff's proposed direction and this proposed fix?

MC: I hear none.

[2019-11 Moved to Ready on Friday AM in Belfast]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify "Table 57 — Meaning of *align* options" [tab:format.align] as indicated:

Table 57 — Meaning of <i>align</i> options [tab:format.align]	
Option	Meaning
<	Forces the field to be left-aligned within aligned to the start of the available space. This is the default for non-arithmetic types, charT, and bool, unless an integer presentation type is specified.
>	Forces the field to be right-aligned within aligned to the end of the available space. This is the default for arithmetic types other than charT and bool or when an integer presentation type is specified.
[...]	

[3329^{\(i\)}](#). **totally_ordered_with both directly and indirectly requires common_reference_with**

Section: 18.5.4 [\[concept.totallyordered\]](#) Status: [Ready](#) Submitter: United States Opened: 2019-11-07 Last modified: 2019-12-02

Priority: 0

View other [active issues](#) in [concept.totallyordered].

View all other [issues](#) in [concept.totallyordered].

View all issues with [Ready](#) status.

Discussion:

Addresses [US 201](#)

The `totally_ordered_with<T, U>` redundantly requires both `common_reference_with<const remove_reference_t&, const remove_reference_t&>` and `equality_comparable_with<T, U>` (which also has the `common_reference_with` requirement). The redundant requirement should be removed.

Proposed change:

Change the definition of `totally_ordered_with` to:

```
template<class T, class U>
concept totally_ordered_with =
    totally_ordered<T> && totally_ordered<U> &&
    equality_comparable_with<T, U> &&
    totally_ordered<
        common_reference_t<
            const remove_reference_t<T>&,
            const remove_reference_t<U>&>> &&
    requires(const remove_reference_t<T>& t,
              const remove_reference_t<U>& u) {
    [... as before ...]
```

[2019-11 Moved to Ready on Friday AM in Belfast]

Proposed resolution:

This wording is relative to [N4835](#).

1. Change 18.5.4 [\[concept.totallyordered\]](#) as indicated:

For some type `T`, let `a`, `b`, and `c` be lvalues of type `const remove_reference_t<T>`. `T` models `totally_ordered` only if

(1.1) — Exactly one of `bool(a < b)`, `bool(a > b)`, or `bool(a == b)` is true.

(1.2) — If `bool(a < b)` and `bool(b < c)`, then `bool(a < c)`.

(1.3) — `bool(a > b) == bool(b < a)`.

(1.4) — `bool(a <= b) == !bool(b < a)`.

(1.5) — `bool(a >= b) == !bool(a < b)`.

```
template<class T, class U>
concept totally_ordered_with =
    totally_ordered<T> && totally_ordered<U> &&
common_reference_with<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
    equality_comparable_with<T, U> &&
    totally_ordered<
        common_reference_t<
            const remove_reference_t<T>&,
            const remove_reference_t<U>&>> &&
equality_comparable_with<T, U> &&
    requires(const remove_reference_t<T>& t,
              const remove_reference_t<U>& u) {
        { t < u } -> boolean;
        { t > u } -> boolean;
        { t <= u } -> boolean;
        { t >= u } -> boolean;
        { u < t } -> boolean;
        { u > t } -> boolean;
        { u <= t } -> boolean;
        { u >= t } -> boolean;
    };
```

[3330^{\(i\)}](#). Include `<compare>` from most library headers

Section: 17.12.1 [\[coroutine.syn\]](#), 19.5.1 [\[system.error.syn\]](#), 20.2.1 [\[utility.syn\]](#), 20.5.2 [\[tuple.syn\]](#), 20.6.2 [\[optional.syn\]](#), 20.7.2 [\[variant.syn\]](#), 20.10.2 [\[memory.syn\]](#), 20.17.1 [\[type.index.synopsis\]](#), 21.3.1 [\[string.syn\]](#), 21.4.1 [\[string.view.synop\]](#), 22.3.2 [\[array.syn\]](#), 22.3.3 [\[deque.syn\]](#), 22.3.4 [\[forward.list.syn\]](#), 22.3.5 [\[list.syn\]](#), 22.3.6 [\[vector.syn\]](#), 22.4.2 [\[associative.map.syn\]](#), 22.4.3 [\[associative.set.syn\]](#), 22.5.2 [\[unord.map.syn\]](#), 22.5.3 [\[unord.set.syn\]](#), 22.6.2 [\[queue.syn\]](#), 22.6.3 [\[stack.syn\]](#), 23.2 [\[iterator.synopsis\]](#), 24.2 [\[ranges.syn\]](#), 27.2 [\[time.syn\]](#), 29.11.5 [\[fs.filesystem.syn\]](#), 30.4 [\[re.syn\]](#), 32.4.1 [\[thread.syn\]](#) **Status:** [Ready](#) **Submitter:** United States **Opened:** 2019-11-07 **Last modified:** 2019-12-02

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses [US 181](#)

The spaceship operator<=> is typically not usable unless the library header <compare> is directly included by the user. Many standard library headers provide overloads for this operator. Worse, several standard classes have replaced their existing definition for comparison operators with a reliance on the spaceship operator, and existing code will break if the necessary header is not (transitively) included. In a manner similar to the mandated library headers transitively #include-ing <initializer_list> in C++11, these headers should mandate a transitive #include <compare>.

Proposed change:

Add:

```
#include <compare>
```

to the header synopsis for each of the following headers:

```
<array>
<chrono>
<coroutine>
<deque>
<forward_list>
<filesystem>
<iterator>
<list>
<map>
<memory>
<optional>
<queue>
<ranges>
<regex>
<set>
<stack>
<string>
<string_view>
<system_error>
<thread>
<tuple>
<type_index>
<unordered_map>
<unordered_set>
<utility>
<variant>
<vector>
```

[2019-11 Moved to Ready on Friday AM in Belfast]

Proposed resolution:

This wording is relative to [N4835](#).

1. Add

```
#include <compare>
```

to the following header synopses:

1. 17.12.1 [\[coroutine.syn\]](#)
2. 19.5.1 [\[system.error.syn\]](#)
3. 20.2.1 [\[utility.syn\]](#)
4. 20.5.2 [\[tuple.syn\]](#)
5. 20.6.2 [\[optional.syn\]](#)
6. 20.7.2 [\[variant.syn\]](#)
7. 20.10.2 [\[memory.syn\]](#)
8. 20.17.1 [\[type.index.synopsis\]](#)
9. 21.3.1 [\[string.syn\]](#)
10. 21.4.1 [\[string.view.synop\]](#)
11. 22.3.2 [\[array.syn\]](#)
12. 22.3.3 [\[deque.syn\]](#)
13. 22.3.4 [\[forward.list.syn\]](#)

14. 22.3.5 [[list.syn](#)]
15. 22.3.6 [[vector.syn](#)]
16. 22.4.2 [[associative.map.syn](#)]
17. 22.4.3 [[associative.set.syn](#)]
18. 22.5.2 [[unord.map.syn](#)]
19. 22.5.3 [[unord.set.syn](#)]
20. 22.6.2 [[queue.syn](#)]
21. 22.6.3 [[stack.syn](#)]
22. 23.2 [[iterator.synopsis](#)]
23. 24.2 [[ranges.syn](#)]
24. 27.2 [[time.syn](#)]
25. 29.11.5 [[fs.filesystem.syn](#)]
26. 30.4 [[re.syn](#)]
27. 32.4.1 [[thread.syn](#)]

3331⁽ⁱ⁾. Define `totally_ordered/_with` in terms of *partially-ordered-with*

Section: 18.5.4 [[concept.totallyordered](#)] **Status:** [Ready](#) **Submitter:** Great Britain **Opened:** 2019-11-08 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in [[concept.totallyordered](#)].

View all other [issues](#) in [[concept.totallyordered](#)].

View all issues with [Ready](#) status.

Discussion:

Addresses [GB 202](#)

Define `totally_ordered[_with]` in terms of *partially-ordered-with*. This will simplify the definition of both concepts (particularly `totally_ordered_with`), and make them in-line with `equality_comparable[_with]`. Now that we've defined *partially-ordered-with* for 17.11.4 [[cmp.concept](#)], we should consider utilising it in as many locations as possible.

Proposed change:

```
template<class T>
    concept totally_ordered =
        equality_comparable<T> &&
        partially_ordered_with<T, T>;

template<class T, class U>
    concept totally_ordered_with =
        totally_ordered<T> &&
        totally_ordered<U> &&
        common_reference_with<
            const remove_reference_t<T>&,
            const remove_reference_t<U>&> &&
        totally_ordered<
            common_reference_t<
                const remove_reference_t<T>&,
                const remove_reference_t<U>&> &&
            equality_comparable_with<T, U> &&
        partially_ordered_with<T, U>;
```

LWG discussion in Belfast notes that [3329](#) also touches the definition of `totally_ordered_with`; the two sets of changes are consistent.

[2019-11 Status to Ready Friday afternoon LWG in Belfast]

Proposed resolution:

This wording is relative to [N4835](#).

1. Change 18.5.4 [[concept.totallyordered](#)] as follows:

```
template<class T>
    concept totally_ordered =
        equality_comparable<T> && partially_ordered_with<T, T>;
        requires(const remove_reference_t<T>& a,
            const remove_reference_t<T>& b) {
            { a < b } -> boolean;
            { a > b } -> boolean;
```

```

{ a <= b } -> boolean;
{ a >= b } -> boolean;
}

```

-1- For some type T , let a , b , and c be lvalues of type `const remove_reference_t< $T$ >`. T models `totally_ordered` only if

(1.1) — Exactly one of `bool(a < b)`, `bool(a > b)`, or `bool(a == b)` is true.

(1.2) — If `bool(a < b)` and `bool(b < c)`, then `bool(a < c)`.

~~(1.3) — `bool(a > b) == bool(b < a)`;~~

(1.4) — `bool(a <= b) == !bool(b < a)`.

(1.5) — `bool(a >= b) == !bool(a < b)`.

```

template<class T, class U>
concept totally_ordered_with =
    totally_ordered<T> && totally_ordered<U> &&
    common_reference_with<const remove_reference_t<T>&, const remove_reference_t<U>&&
    totally_ordered<
        common_reference_t<
            const remove_reference_t<T>&,
            const remove_reference_t<U>&>> &&
    equality_comparable_with<T, U> &&
    partially_ordered_with<T, U>;
    requires(const remove_reference_t<T>& t,
        const remove_reference_t<U>& u) {
        { t < u } -> boolean;
        { t > u } -> boolean;
        { t <= u } -> boolean;
        { t >= u } -> boolean;
        { u < t } -> boolean;
        { u > t } -> boolean;
        { u <= t } -> boolean;
        { u >= t } -> boolean;
    }
}

```

[...]

3332⁽ⁱ⁾. Issue in `[time.format]`

Section: 27.12 [\[time.format\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Mateusz Pusz **Opened:** 2019-11-05 **Last modified:** 2019-12-02

Priority: 0

View other [active issues](#) in `[time.format]`.

View all other [issues](#) in `[time.format]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

Table 97 `[tab:time.format.spec]` enumerates 'q' and 'Q' but those are not specified in *chrono-format-spec* provided in paragraph 1 of sub-clause 27.12 [\[time.format\]](#).

[2019-11-23 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4835](#).

1. Change the *type* grammar element in 27.12 [\[time.format\]](#) p1 as indicated:

```

type: one of
    a A b B c C d D e F g G h H I j m M n
    p q_Q r R S t T u U V w W x X y Y z Z %

```

3338⁽ⁱ⁾. Rename `default_constructible` to `default_initializable`

Section: 18.4.12 [\[concept.defaultconstructible\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-11-18 **Last modified:** 2019-12-02

Priority: 0

View all other [issues](#) in [\[concept.defaultconstructible\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

WG21 merged [P1754R1](#) "Rename concepts to standard_case for C++20" into the working draft as LWG Motion 11 in 2019 Cologne. That proposal contains editorial instructions to rename what was the `DefaultConstructible` concept:

```
IF LWG3151 ACCEPTED:
    default_initializable
ELSE
    default_constructible
```

Notably LWG [3151](#) "ConvertibleTo rejects conversions from array and function types" is not the intended issue number, LWG [3149](#) "DefaultConstructible should require default initialization" is. It was made clear during discussion in LEWG that 3149 would change the concept to require default-initialization to be valid rather than value-initialization which the `is_default_constructible` trait requires. LEWG agreed that it would be confusing to have a trait and concept with very similar names yet slightly different meanings, and approved [P1754R1](#)'s proposed renaming.

LWG [3149](#) was moved to "Ready" but not approved by WG21 until Belfast — after the application of P1754R1 to the working draft — so this renaming has not happened, but the rationale remains valid.

[2019-11-30 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4835](#).

1. Change the stable name "[concept.defaultconstructible]" to "[concept.default.init]" and retitle "Concept default_constructible" to "Concept default_initializable". Replace all references to the name `default_constructible` with `default_initializable` (There are 20 occurrences).

[3346^{\(i\)}](#). `pair` and `tuple` copy and move constructor have backwards specification

Section: 20.4.2 [\[pairs.pair\]](#), 20.5.3.1 [\[tuple.cnstr\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2019-11-26 **Last modified:** 2019-12-09

Priority: 0

View other [active issues](#) in [\[pairs.pair\]](#).

View all other [issues](#) in [\[pairs.pair\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

20.4.2 [\[pairs.pair\]](#) p2 and 20.5.3.1 [\[tuple.cnstr\]](#) p3 say:

The defaulted move and copy constructor, respectively, of `{pair,tuple}` is a constexpr function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a constexpr function.

Note that we specify the copy constructor in terms of element move operations and the move constructor in terms of element copy operations. Is that really the intent? This appears to be how this was originally specified when the wording was added by [N3471](#).

[2019-12-01; Daniel comments and provides wording]

These inverted wording effects are an unintended oversight caused by [N3471](#).

[2019-12-08 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after ten positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 20.4.2 [\[pairs.pair\]](#) as indicated:

-2- The defaulted move and copy constructor, respectively, of pair shall be a constexpr function if and only if all required element-wise initializations for `copymove` and `movecopy`, respectively, would satisfy the requirements for a constexpr function.

2. Modify 20.5.3.1 [\[tuple.cnstr\]](#) as indicated:

-3- The defaulted move and copy constructor, respectively, of tuple shall be a constexpr function if and only if all required element-wise initializations for `copymove` and `movecopy`, respectively, would satisfy the requirements for a constexpr function. The defaulted move and copy constructor of tuple<> shall be constexpr functions.

[3349^{\(i\)}](#). Missing `__cpp_lib_constexpr_complex` for P0415R1

Section: 17.3.2 [\[version.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2019-12-04 **Last modified:** 2020-01-12

Priority: 0

View other [active issues](#) in [\[version.syn\]](#).

View all other [issues](#) in [\[version.syn\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

[P1902R1](#) "Missing feature-test macros 2017-2019", accepted in Belfast, said:

[\[P0415R1\]](#) (Constexpr for `std::complex`): this paper proposes to introduce the macro `__cpp_lib_constexpr_complex`. That is, introducing a new macro for this header.

However, `__cpp_lib_constexpr_complex` wasn't mentioned in the Wording section, and doesn't appear in the latest WP [N4842](#).

[P0415R1](#) was accepted in Albuquerque (November 2017), and the paper itself said "For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_constexpr_complex`.", so this has been accepted and then overlooked by WG21 twice.

[2019-12-12 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after twelve positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 17.3.2 [\[version.syn\]](#) p2 as indicated:

```
[...]  
#define __cpp_lib_constexpr_algorithms 201806L // also in <algorithm>  
#define __cpp_lib_constexpr_complex 201711L // also in <complex>  
#define __cpp_lib_constexpr_dynamic_alloc 201907L // also in <memory>  
[...]
```

[3350^{\(i\)}](#). Simplify return type of `lexicographical_compare_three_way`

Section: 25.7.11 [\[alg.three.way\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-12-04 **Last modified:** 2020-01-12

Priority: 0

View all other [issues](#) in [\[alg.three.way\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The current return type is:


```
common_comparison_category_t<decltype(comp(*b1, *b2)), strong_ordering>
```

Finding the common category with `strong_ordering` doesn't do anything. The common category of `X` and `strong_ordering` is always `X`, so we can simplify it to:

```
common_comparison_category_t<decltype(comp(*b1, *b2))>
```

This can further be simplified, because the common category of any comparison category type is just that type. If it's not a comparison category then the result would be void, but the function would be ill-formed in that case anyway, as we have:

Mandates: `decltype(comp(*b1, *b2))` is a comparison category type.

So the only effect of the complicated return type seems to be to cause the return type to be deduced as void for specializations of the function template that are ill-formed if called. That doesn't seem useful.

[2019-12-12 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 25.4 [\[algorithm.syn\]](#) as indicated:

```
[...]
// 25.7.11 \[alg.three.way\], three-way comparison algorithms
template<class InputIterator1, class InputIterator2, class Cmp>
constexpr auto
    lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                      InputIterator2 b2, InputIterator2 e2,
                                      Cmp comp)
    -> common_comparison_category_tdecltype(comp(*b1, *b2)), strong_ordering;
template<class InputIterator1, class InputIterator2>
constexpr auto
    lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                      InputIterator2 b2, InputIterator2 e2);
[...]
```

2. Modify 25.7.11 [\[alg.three.way\]](#) as indicated:

```
template<class InputIterator1, class InputIterator2, class Cmp>
constexpr auto
    lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                      InputIterator2 b2, InputIterator2 e2,
                                      Cmp comp)
    -> common_comparison_category_tdecltype(comp(*b1, *b2)), strong_ordering;

-1- Mandates: decltype(comp(*b1, *b2)) is a comparison category type.

[...]
```

[3351^{\(i\)}](#). `ranges::enable_safe_range` should not be constrained

Section: 24.2 [\[ranges.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-12-05 **Last modified:** 2020-01-12

Priority: 0

View other [active issues](#) in `[ranges.syn]`.

View all other [issues](#) in `[ranges.syn]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

Currently `ranges::enable_safe_range` is constrained with `ranges::range`, which not only forces the compiler to do unnecessary satisfaction checking when it's used, but also creates a tricky dependency cycle (`ranges::range` depends on `ranges::begin` which depends on `ranges::enable_safe_range` which depends on `ranges::range`).

The only place the variable template is expected to be used is in the `ranges::safe_range` concept, which already checks `range<T>` before using `enable_safe_range<T>` anyway.

The constraint serves no purpose and should be removed.

[2019-12-12 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after eight positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 24.2 [\[ranges.syn\]](#) as indicated:

[Drafting note: The definition in 24.4.2 [\[range.range\]](#) p7 is already unconstrained, which contradicts the synopsis.]

```
[...]
// 24.4.2 \[range.range\], ranges
template<class T>
concept range = see below;

template<rangeclass T>
inline constexpr bool enable_safe_range = false;

template<class T>
concept safe_range = see below;
[...]
```

[3356^{\(i\)}](#). `__cpp_lib_nothrow_convertible` should be `__cpp_lib_is_nothrow_convertible`

Section: 17.3.2 [\[version.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Barry Revzin **Opened:** 2019-12-09 **Last modified:** 2020-01-12

Priority: 0

View other [active issues](#) in [\[version.syn\]](#).

View all other [issues](#) in [\[version.syn\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

[P1902R1](#) introduced the feature test macro `__cpp_lib_nothrow_convertible`, but every other example in SD-6 of a feature test macro testing for the presence of a single type trait *FOO* is named `__cpp_lib_FOO`. This macro should be renamed `__cpp_lib_is_nothrow_convertible`.

This naming convention should probably be documented in SD-6 as policy.

[2019-12-21 Issue Prioritization]

Status to Tentatively Ready and priority to 0 after seven positive votes on the reflector. A convincing argument was that currently no vendor had published a release with the previous feature macro.

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, as indicated:

```
[...]
#define __cpp_lib_not_fn                201603L // also in <functional>
#define __cpp_lib_is_nothrow_convertible 201806L // also in <type_traits>
#define __cpp_lib_null_iterators       201304L // also in <iterator>
[...]
```

[3360^{\(i\)}](#). `three_way_comparable_with` is inconsistent with similar concepts

Section: 17.11.4 [\[cmp.concept\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-12-18 **Last modified:** 2020-01-12

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The concept `three_way_comparable_with` is defined in 17.11.4 [\[cmp.concept\]](#) as:

```
template<class T, class U, class Cat = partial_ordering>
concept three_way_comparable_with =
    weakly-equality-comparable-with<T, U> &&
    partially-ordered-with<T, U> &&
    three_way_comparable<T, Cat> &&
    three_way_comparable<U, Cat> &&
    common_reference_with<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
    three_way_comparable<
        common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>, Cat> &&
    requires(const remove_reference_t<T>& t, const remove_reference_t<U>& u) {
        { t <=> u } -> compares-as<Cat>;
        { u <=> t } -> compares-as<Cat>;
    };
```

Which notably doesn't follow the requirement ordering:

1. same-type requirements on T
2. same-type requirements on U
3. `common_reference_with` requirement
4. same-type requirements on `common_reference_t<T, U>`
5. cross-type requirements on T and U

that the other cross-type comparison concepts (18.5.3 [\[concept.equalitycomparable\]](#), 18.5.4 [\[concept.totallyordered\]](#)) use. There were some motivating reasons for that ordering:

1. The existence of a common reference type is effectively an opt-in to cross-type concepts. Avoiding checking cross-type expressions when no common reference type exists can enable the concepts to work even in the presence of poorly-constrained "accidental" cross-type operator templates which could otherwise produce compile errors instead of dissatisfied concepts.
2. Putting the simpler same-type requirements first can help produce simpler error messages when applying the wrong concept to a pair of types, or the right concept to the wrong pair of types. "Frobnozzle <=> Frobnozzle is not a valid expression" is more easily deciphered than is "std::common_reference<int, FrobNozzle> has no member named type".

`three_way_comparable_with` should be made consistent with `equality_comparable_with` and `totally_ordered_with` for the above reasons and to make it easier to reason about comparison concepts in general.

[01-2020 Status set to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 17.11.4 [\[cmp.concept\]](#) as indicated:

```
template<class T, class U, class Cat = partial_ordering>
concept three_way_comparable_with =
    weakly-equality-comparable-with<T, U> &&
    partially-ordered-with<T, U> &&
    three_way_comparable<T, Cat> &&
    three_way_comparable<U, Cat> &&
    common_reference_with<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
    three_way_comparable<
        common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>, Cat> &&
    weakly-equality-comparable-with<T, U> &&
    partially-ordered-with<T, U> &&
    requires(const remove_reference_t<T>& t, const remove_reference_t<U>& u) {
        { t <=> u } -> compares-as<Cat>;
        { u <=> t } -> compares-as<Cat>;
    };
```